Log to Incident

Gary Brooks

2026-07-28

Table of Contents

Chapter 1. Purpose

Large incident resolution times hurt business operations and customer satisfaction. One way to shorten resolution is to bring critical application problems to the right operations team quickly and with enough context to act. That requires bridging **observability** (detecting and describing problems from runtime signals) with **IT service management** (tracking work as incidents, assignment, and closure).

This document defines that bridge as the **Log to Incident** process: selected application log messages become managed incidents for the application operations team. The process is named for the **signal type** and **outcome**, not for any particular vendor or tool. Implementations may use different observability and ITSM platforms; this document's worked example uses Apache Spark as the application, with platform-specific sections for how those platforms are configured.

Chapter 2. Log to Incident

Log to Incident is the Observability process that detects and turns **critical** application log messages into **tracked** IT incidents so the application operations team can respond in time.

2.1. The Process

The Log to Incident process takes one or more log messages and generates an incident in the ITSM platform through a three phase process:

1. The application logs a message
2. The observability tooling detects the message as a being salient and creates an alert and sends it to the ITSM platform
3. The ITSM platform determines which alerts are salient and then creates an incident for operators to track.

Figure 1 illustrates these phases in more detail.

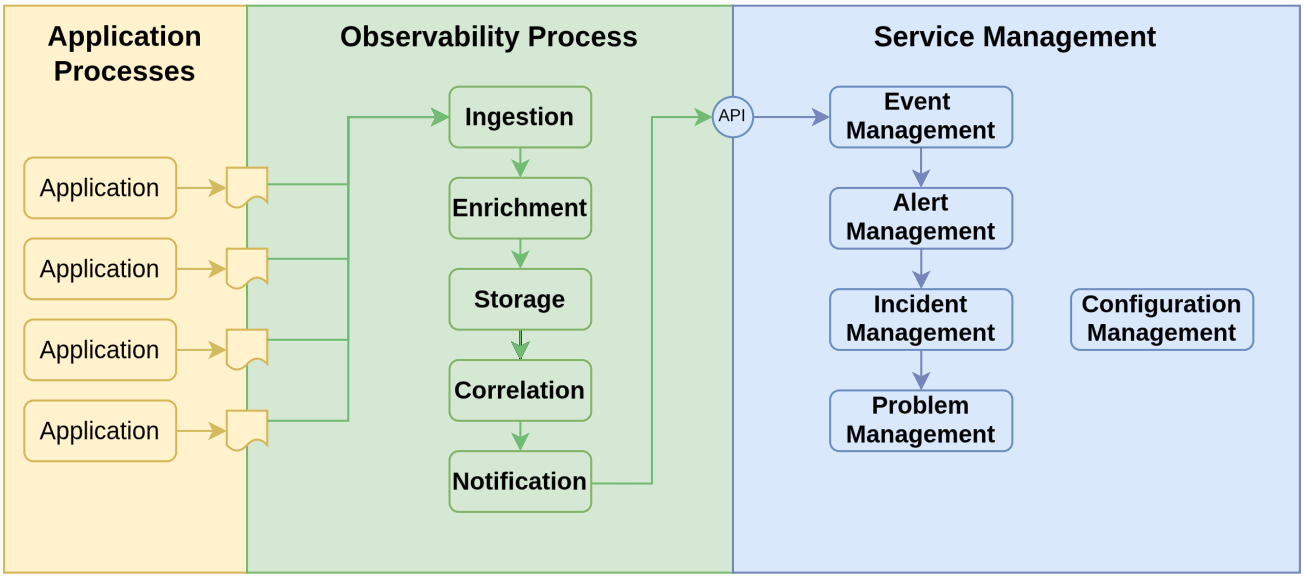

Figure 1. Log to Incident process phases

2.1.1. Phase 1: Application Logging

To various degrees applications are written to generate log messages to a file or other storage medium or APIs. The log messages can indicate forward progress in the application or problems that have occurred. The problems can range from minor inconsistencies to major failures that require immediate attention.

2.1.2. Phase 2: Observability Tooling

The observability platform ingests the log messages either through an agent reading the log files or via an API that the application calls. Either way, the observability platform processes each log message in four addition steps.

1. **Enrichment**: The observability platform adds additional information to the log message. This information comes from the context of the log file (e.g. name of the log file) as well as by parsing information within the log file. This information can include can include the application name, the environment, the host name, the process id, the thread id, the timestamp, the log level, the log message, and the log message id.
2. **Storage**: The observability platform takes the enriched log message and stores it in a database for fast retrieval and further processing.
3. **Correlation**: The observability platform correlates the log message with other log messages or events to group related log messages into a single message to increase the signal to noise ration and to identify patterns and relationships with other log messages or events.
4. **Notification**: The observability platform then takes selected messages or message groups and`` sends the enriched log message to the ITSM platform as an event.

Coordination between the application developers and the observability platform engineers. Ideally, this collaboration would begin in the application design phase to ensure that the log messages are structured in a way that that simplifies the enrichment process and identification of critical log messages.

2.1.3. Phase 3: ITSM Platform

The ITSM platform accepts events from the observability platform and determines which events (aka log messages in this cae) are salient and then creates an incident for operators to track. This happens in a three stage process with a fourth follow on stage to handle multiple incidents that are related to the same underlying problem.

1. **Event Management**: Event management accepts an incoming event (log message) from the observability platform. Events may be clear events that indicate a past issue has been resolved. Or they may be events that indicate a new issue has been detected. Salient events are then forwarded to Alert Management.
2. **Alert Management**: Alert Management receives the salient events from Event Management and creates an alert record in the Alert Management platform. Alerts are bound (mapped) to a configuration item (CI) that best represents the application or the infrastructure component that best represents the component causing or involved in the event. Mapping an alert to a CI requires designing the CMdb model and the information in the event so that Alert Management can use the information sent in the event(s) to map the alert to the correct CI. Multiple events may be mapped to a single alert depending on a message key or other criteria. Alert management will take a clear event will close out an existing, an alert with the same message key. Salient alerts are then forwarded to Incident Management.
3. **Incident Management**: Incident Management is where operations teams interact with incidents to resolve the root causes of the incident. When incident management receives a salient alert, it will either bind the alert to an existing incident or create a new incident. The incident will also be bound to a configuration item (CI) that best represents the application or the infrastructure component that best represents the component causing or involved in the event. Mapping an incident to a CI requires designing the CMdb model and the information in the alert so that Incident Management can use the information sent in the alert to map the

incident to the correct CI.

Incident Management will also assign (or determine) the support organization that will be responsible for resolving the incident. The incident will be assigned to the application's support organization and will carry enough log context for diagnosis.

4. **Problem Management**: Problem Management focuses on the root cause analysis of recurring incidents (once service has been restored). A problem will typically map onto several closed incidents. This process is outside the scope of this document.

2.2. Client and server applications

Many systems have both **service-side** components (clustered services, pods, long-running servers) and **client-side** components (drivers, desktops, mobile apps, or other unmanaged endpoints). Clients complicate modeling the application in a CMdb as the client-side components are not managed by the CMdb.

- **Service-side**: map from the running workload's identity to the application service that operations supports.
- **Client-side**: map from a durable log-path or naming contract to a logical application service that represents the client population.

Log to Incident must still hand both scenarios, even when the client is not a managed CI in the CMDB.

2.3. Design considerations

Specific observability platforms (e.g. Dynatrace or Elastic Stack) and specific ITSM platforms (e.g. ServiceNow or Remedy) will have specific requirements for implementing the Log to Incident observability process. However, there are the several design considerations that span the different platforms. These include:

- Which log messages are considered salient and should be turned into an incident?
- How to model the (service-side) application in the CMdb?
- How to model clients in the CMdb?
- What information must an event, whether from a client or a service, include to determine the CI to map to an incident?

2.4. Platform implementations

Subsequent chapters describe how to implement the Log to Incident process on specific observability and service management platforms. To design and configure such systems, we need to have a specific application in mind that we will use to illustrate the process. For historical reasons, we will use Apache Spark as the application. Apache Spark is a sophisticated data and analytics platform that runs on Kubernetes and has both client and and service use cases.

Information on Apache Spark is available at Apache Spark Documentation.

Chapter 3. Spark Architecture

Before we discuss how to observe and manage an application, we need to understand the architecture of the application. Spark is a Kuberenetes application. The Spark application consists of a Spark Master (Driver) and one or more Spark Workers. The Spark Master is the central point of control for the Spark application. The Spark Workers are the nodes that do the actual work of the application. You can scale the Spark Workers horizontally or vertically depending on available resources. In addition, there is a Spark History Server that is used to store the history of the application. The History Server is a built-in observability components that provides debugging and monitoring capabilities for Spark. The executors on the workers run jobs and constitutes the heart of the application.

Spark can run in one of two modes: **Client Mode** or **Cluster Mode**. In cluster mode all components run on the Kubernetes cluster. In cluster mode, the Spark Driver runs on a Kubernetes pod and communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side. Figure 2 illustrates the cluster mode architecture.

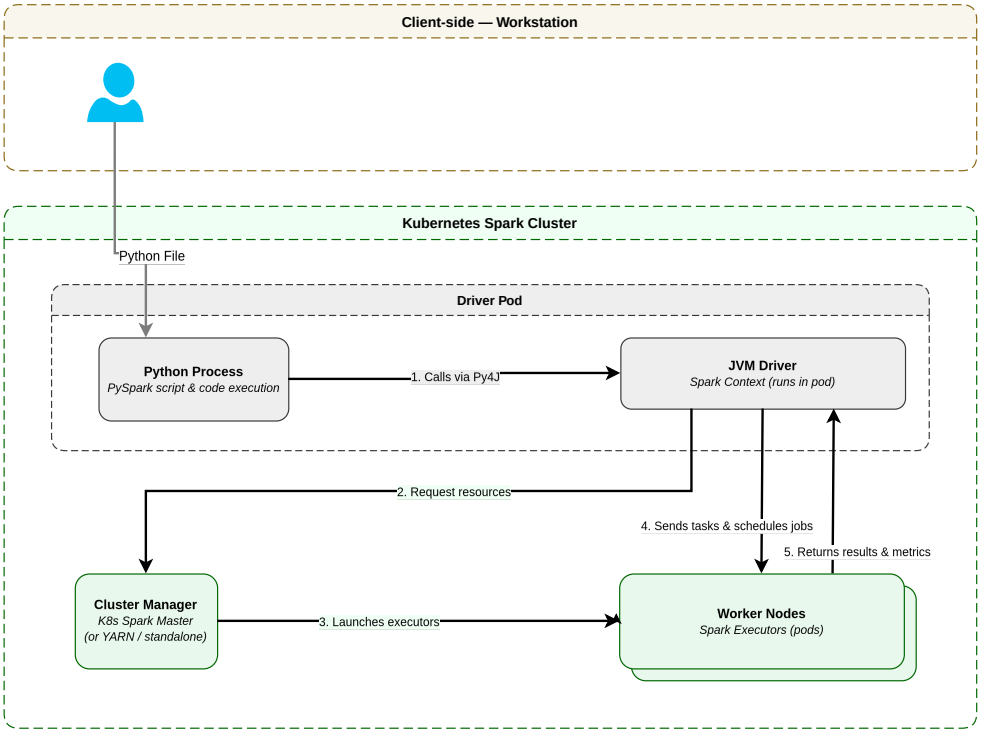

Figure 2. Spark Cluster Mode Architecture

In cluster mode, the users interacts with Spark by submitting a PySpark program to the Spark Driver. This is essentially a batch submission. The Spark Driver then communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side. The executors on the workers run jobs and constitutes the heart of the application.

Client mode is a client-server architecture where the client-side application is a standalone driver that can run on a workstation. Client mode allows data scientists and engineers to work interactively and iteratively with Spark directly on their own laptops or workstations. client is

shown in the [fig-client-mode-architecture].

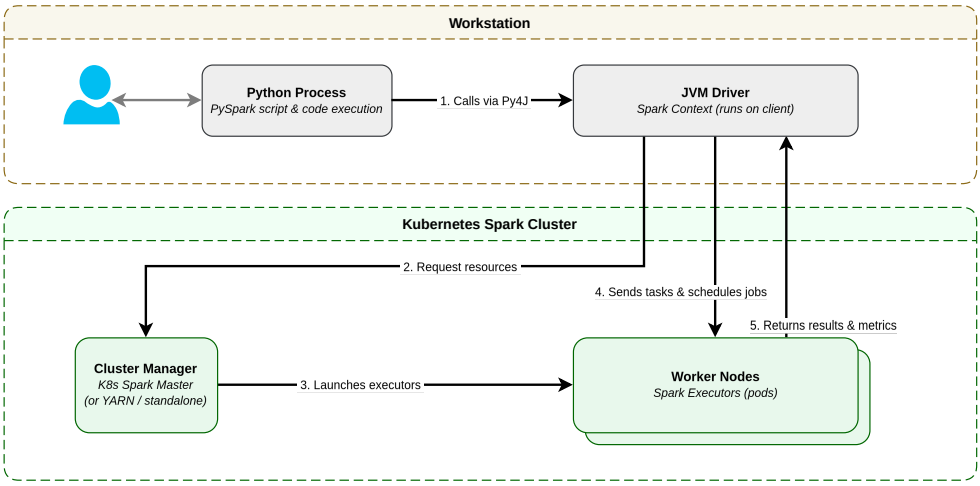

Figure 3. Spark Client Architecture

In client mode, key parts of the Spark Driver run directly on the user's workstation. The user then interacts iteratively with the PySpark process to manipulate Spark data (DataFrames and Datasets) through PySpark programs and Spark SQL queries. The PySpark process then communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side.

Spark inherently supports both Client Mode (client-side) and Cluster Mode (server-side). We have chosen to support both modes within the reference implementation in part because Spark does, but also because, client-side applications occur within many desktop and mobile applications. Client-side applications provide additional challenges in modeling and managing applications.

You can deploy Spark as a client-server application or a server-only application. We had chosen enable the client-server mode to allow data scientists or engineers to launch applications from their own laptops or workstations. This allows for a more flexible and efficient development and testing environment. It also provides a more robust observability exercise as we have to address the client use case, which can occur in many other applications. For example, you can see the client in mobile apps where the client is not only a separate application, but it is an applications that runs on a device that is not managed in ServiceNow's CMDB.

For those not familiar with Spark's client-server model, the Figure 3 illustrates the client architecture showing the different components of the Spark application in client mode.

Spark can run in one of two modes: **Client Mode** or **Cluster Mode**. In client mode, the client-side application is a standalone application that can run on a workstation or a mobile . In some cases, like this one, the "workstation" may also be a host. In all these cases, the client-side application is not managed by ServiceNow's CMDB. In the case of a true workstation, the workstation is not managed in ServiceNow's CMdb.

The client then communicates with the Spark Master running as a service and the executors running on the Spark Workers. The Spark Master and the workers run on the service-side. The executors on the workers run jobs and constitute the heart of the application.

Chapter 4. Dynatrace and ServiceNow

This chapter documents a reference implementation of Log to Incident using **Dynatrace** as the observability platform and **ServiceNow** as the ITSM & Event Management platform used to observe and manage an Apache Spark implementation. The Figure 4, below, illustrates the end-to-end flow log messages from the Spark application through Dynatrace to ServiceNow. This is a for phase process that starts with Spark writing to an application log file; Dynatrace then ingests and processes a log line, deciding which ones are critical issues (Problems) and then notifies ServiceNow of critical issue. ServiceNow ingest the issue (event) and creates an alert and incident mapping bot to appropriate CIs. ServiceNow then notifies the support team of the CI assigned to the incident.

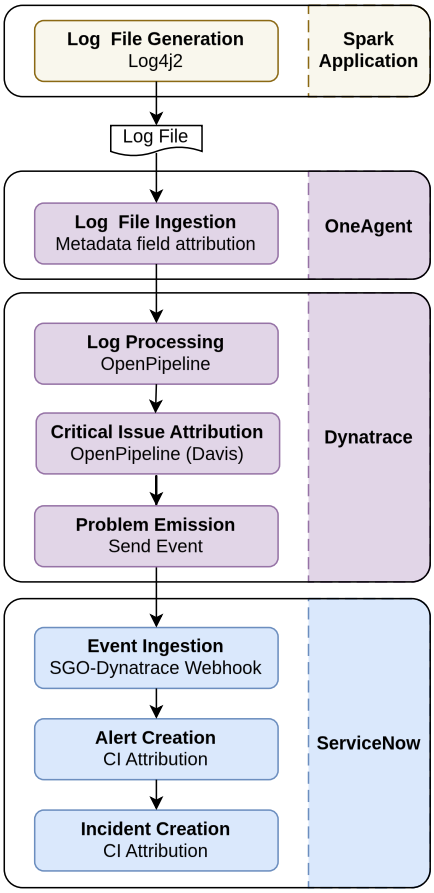

Figure 4. Log to Incident data flow

In ServiceNow, operational issues are managed at the incident level The support group of the CI assigned to an alert or incident is the group that will be notified when an issues arises. If the wrong CI (with an inappropriate support group) is assigned to the incident it will take a longer time to notify the right support team. This makes assigning a CI to an incident critical to the process. For application log messages, the assigned CI should be the application service of the application instance emitting the log message. Finding the right CI at runtime to assign to a log message does not happen automatically. It needs to be designed into the CMdb model of the application.

Designing the CMdb model for the application is another phase in the process. It needs to be done before the application ever runs. The table below enumerates all the major steps required to

configure the Log to Incident flow and the touches on some of the key design issues for the step.

Table 1. Log to Incident configuration phases

Phase	Layer	Design Goals
Phase 1	ServiceNow Modeling	• We need to understand the relationships in the CMdb model and how they relate to runtime components so that at the time a log message (event) is ingested by ServiceNow, we can eventually assign the correct CI to the incident. This in turns understanding the runtime CIs define by Dynatrace and how they relate to the rest of the CMdb model.
Phase 2	Spark Application	• **Log file generation:** Where and how does Spark generate the log files and how does Dynatrace's OneAgent extract useful metadata from them to send to Dynatrace proper?
Phase 3	OneAgent	• **Log file ingestion:** How does OneAgent ingest the log data and extract useful metadata from the log files and context and send them to Dynatrace?
Phase 4	Dynatrace	• **Log Processing:** How does Dynatrace process the data and metadata sent from OneAgent? • **Critical issue determination:** How does Dynatrace determine which log messages are critical and create a Dynatrace problem that should be sent to ServiceNow (as an event)? • **Problem notification:** How does Dynatrace send the Problem (as an event) to ServiceNow?
Phase 5	ServiceNow	• **Event Ingestion:** How does ServiceNow ingest an event from Dynatrace? • **Alert Creation:** How and when does it create an alert for the event and what CI does it assign to the alert? • **Incident Creation:** When an incident is created, how does ServiceNow identify the correct CI to bind to the incident?

Client Mode configuration is Chapter 5; Cluster Mode (stdout / Container Output) is Chapter 6. ServiceNow Event Management remains Phase 4.

Chapter 5. Phase 1 — ServiceNow Modeling

The first Phase in the process is to model the application in ServiceNow. It needs to happen even before the application ever runs. The ServiceNow modeling is the data foundation of the Log to Incident process. It consists of the CSDM hierarchy and the pattern-specific CMDB components. We need to do this for the Service-side and the Client-side components.

Before we can bind incidents in ServiceNow to a an appropriate CI, we must model the application in ServiceNow, including both the client-side and service-side use cases. Furthermore, we need to model the application in a manner that follows ServiceNow's CSDM best practices. This involves defining the CSDM hierarchy and discovering pattern-specific CMDB components. We need to do this for the Service-side and the Client-side Spark use cases.

In these models, some CIs (of components) are automatically discovered by ServiceNow and other, typically in the CSDM model, are manually defined to represent the application within an enterprise. Other CIs are imported from other systems, like Dynatrace.

We first examine the CSDM model that is common to both the client and service-side applications. In the client-side scenario we have both a client and service-side application. As such we examine the service-side model next and then the client-side model after that.

5.1. Common CSDM Model

The CSDM is part of the overall ServiceNow model for an application. It models how an application fits within the enterprise. It is a hierarchy of CIs that starts with the application service, which models the application instance. Associated with the application service at a lower (conceptual) level are the CIs that represent the application's Kubernetes components. The Business Application (cmdb_ci_service) represents the business' view of the application - i.e. what more generalized, business oriented, service does it provide. In this case, it is an Apache Spark service that data scientists and engineers use to perform data analytics and machine learning. There could be other instances of the Spark Master service in the enterprise. Lastly, the Business Application represents the overall functional capability area. In this case, it is Data and Analytics. There could be other technologies used in the enterprise aside from Apache Spark (e.g. Snowflake), which would get their own Business Service and still fall under the Data and Analytics Business Application.

Figure 5 illustrates the common CSDM model for the Spark application.

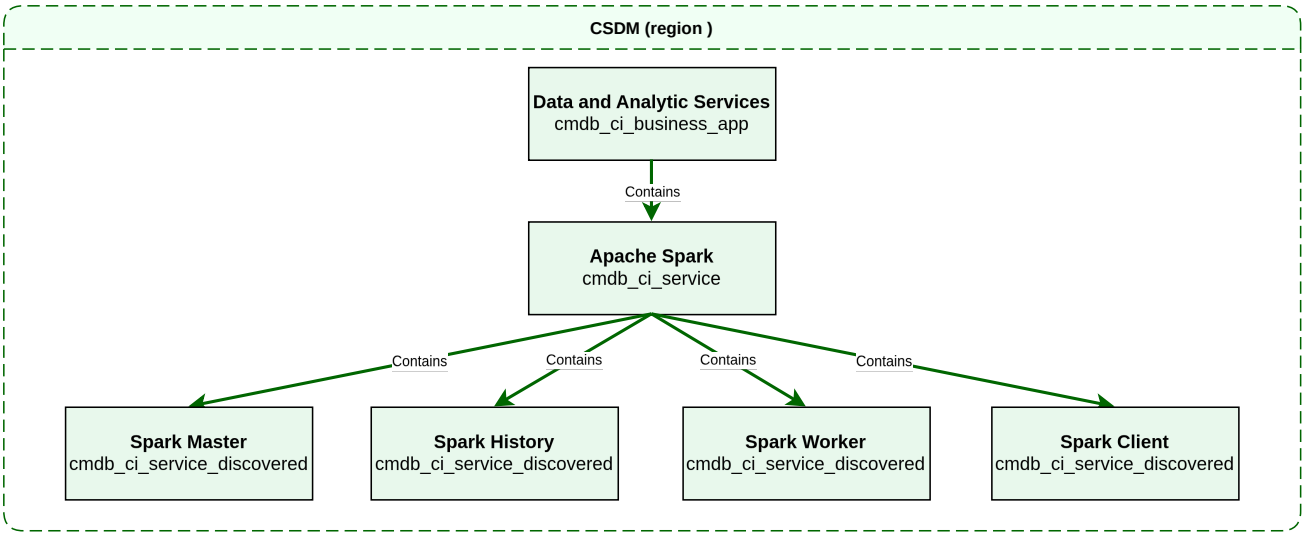

Figure 5. Spark CSDM Model

There is an application service each of the materialized Spark services: the Spark Master, the Spark Workers, and the Spark History Server. The Spark Master is the central point of control for the Spark application. The Spark Workers are the nodes that do the actual work of the application. The Spark History Server is a built-in observability component that provides debugging and monitoring capabilities for Spark. The forth application service is a synthetic application service that represents all Spark clients that talk directly to a the Spark Workers without a using a Spark Master. We will discuss this further in the Figure 7 section.

You must manually create each one of the CSDM CIs in ServiceNow. To simplify the tedium of creating the CSDM CIs, we have used a tool to create the CIs based on a YAML specification. While the tool is beyond the scope of this document we share the YAML specification in Listing 1, below. It illustrates most of the key CI attributes needed for the Spark application.

Listing 1. Spark CSDM Model for the Spark Master

```
business_applications:
  - name: Data and Analytic Services
    identifier: data-and-analytic-services
    short_description: Data processing and analytics platform services
    operational_status: "1"
    active: "true"
    business_owner: gbrooks
    it_application_owner: gbrooks

business_services:
  - name: Apache Spark
    identifier: apache-spark
    short_description: Distributed data processing cluster
    operational_status: "1"
    parent_business_application: Data and Analytic Services
    owned_by: gbrooks
    business_criticality: "4 - not critical"

service_instances:
  - name: Spark Master
    identifier: spark-master
    short_description: Spark master — Kubernetes StatefulSet
    operational_status: "1"
    parent_business_service: Apache Spark
    platform: kubernetes
    service_mapping: tags
```

```
    discover: false
    environment: on-prem
    location: brooks-lab
    owned_by: gbrooks
    business_criticality: "4 - not critical"
    depends_on:
      - OpenTelemetry Collector
      - Logstash
      - Elasticsearch
      - name: lab3
        type: nfs_server
```

5.2. Service-Side Model

The Service-side model used in Spark cluster mode, shows the logical relationships between CSDM Application Services, Kubernetes-discovered CIs, SGC-imported entities, and Event Management bindings for pod-scoped (server side)logs. One of the main outcomes of this model is to be able to map service-side alerts to a reasonable CI and incidents to an application service CI for more timely communication to the appropriate support team.

Figure 6 illustrates the Service-side model of the Spark Master. The other Spark services, e.g. Spark Workers and the Spark History Server, would have avery similar CMdb models.

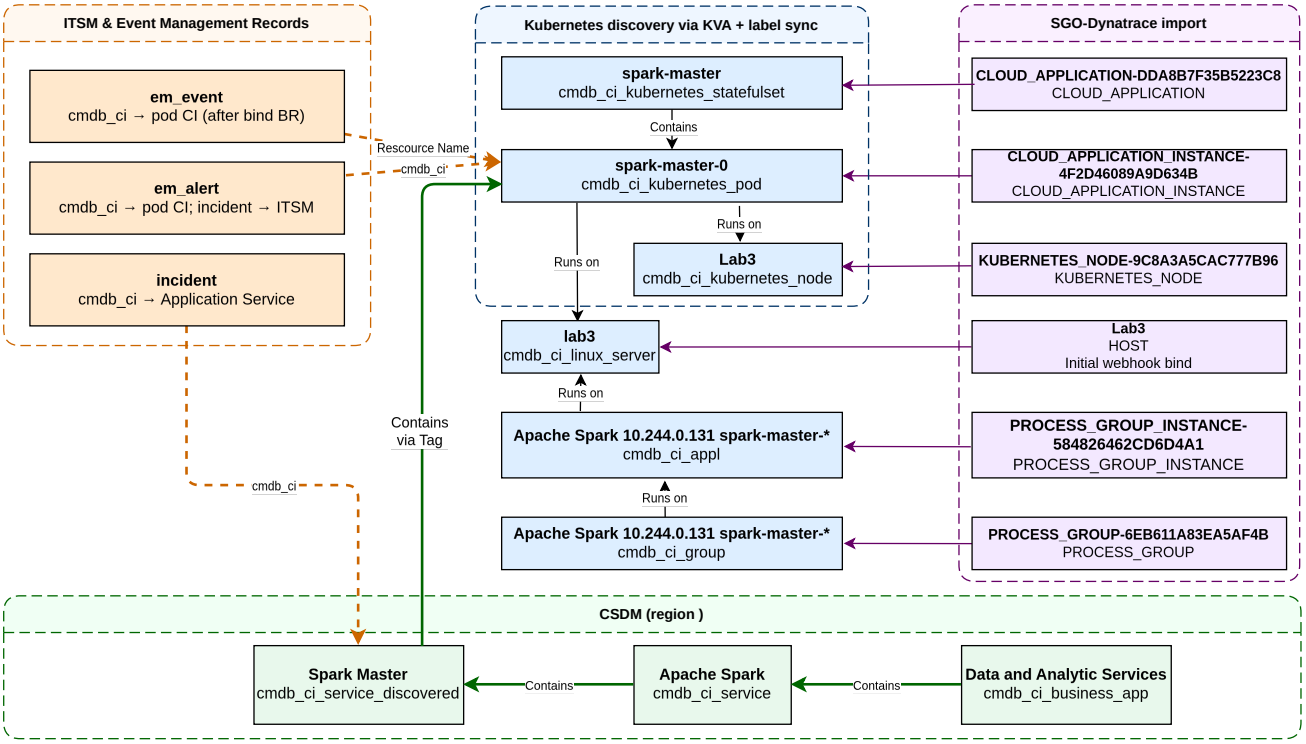

Figure 6. Spark Service-Side Model

The table below enumerates the different CMDB objects in the Service-side scenario. For those CIs that Dynatrace can generate or that you can correlate to a CI, the corresponding Dynatrace entity type is shown. The last column summarizes how Service Graph Connector (SGC / **SGO-Dynatrace**) correlates the Dynatrace entity to the CMDB CI. Key in this process is what the Identity and Reconciliation Engine (IRE) does to merge the Dynatrace entity with the CMDB CI.

Table 2. Service-side CMDB objects and SGC correlation

CI Name	CI Class	DT Entity	SGC correlation to CI
spark-master	cmdb_ci_kubernetes_statefulset	CLOUD_APPLICATION	SGC Kubernetes workload import maps `CLOUD_APPLICATION` → workload CI; `entityId` stored in `sys_object_source` (`name=SGO-Dynatrace`). **IRE merges with the KVA StatefulSet when names match.**
spark-master-0	cmdb_ci_kubernetes_pod	CLOUD_APPLICATION_INSTANCE	**SGO-Dynatrace Kubernetes Pod** feed maps `CLOUD_APPLICATION_INSTANCE` → `cmdb_ci_kubernetes_pod`; `entityId` → `sys_object_source.id`. **IRE merges with the KVA pod when `name` matches.**
lab3	cmdb_ci_kubernetes_node	KUBERNETES_NODE	**SGO-Dynatrace Kubernetes Node** feed maps `KUBERNETES_NODE` → `cmdb_ci_kubernetes_node`; `entityId` → `sys_object_source.id`. Merges with the KVA node CI on node name.
lab3	cmdb_ci_linux_server	HOST	**SGO-Dynatrace Hosts** feed maps `HOST` → computer / `cmdb_ci_linux_server`; `entityId` → `sys_object_source.id`. **IRE merges with Horizontal Discovery on a case insensitive match of hostname or FQDN.**
Apache Spark 10.244.0.131 spark-master-* - PROCESS_GROUP -6EB611A83EA5AF4B	cmdb_ci_group	PROCESS_GROUP	**SGO-Dynatrace Process Groups** feed maps `PROCESS_GROUP` → `cmdb_ci_group`; `entityId` → `sys_object_source.id`.
Apache Spark 10.244.0.131 spark-master-* spark-master-* spark-master@lab3	cmdb_ci_appl	PROCESS_GROUP_INSTANCE	**SGO-Dynatrace** process / PGI cascade maps `PROCESS_GROUP_INSTANCE` → `cmdb_ci_appl`; `entityId` → `sys_object_source.id`.

To be able to correlate a critical log message to the appropriate application service, we need to use tag-based service mapping to correlate the Kubernetes pod to the application service. This correlation is accomplished by setting the `servicenow.com/service-instance` label in the template for the Kubernetes pod. The label value is the **display name** of the service instance. Because Kubernetes label values cannot contain spaces or parentheses, the display name is written in the Kubernetes label character set — spaces become hyphens (`Spark Master` → `Spark-Master`). Docker labels have no such restriction and carry the display name verbatim.

Spark Master StatefulSet pod template for the Spark Master is shown below.

Listing 2. Spark Master StatefulSet pod template labels

```
template:
  metadata:
    labels:
      app: spark-master
```

```
    app.kubernetes.io/name: spark-master
    app.kubernetes.io/instance: brooks-lab
    app.kubernetes.io/component: master
    app.kubernetes.io/part-of: apache-spark
    app.kubernetes.io/managed-by: ansible
    servicenow.com/service-instance: Spark-Master
```

The `servicenow.com/service-instance` label correlates the pod to the service instance. Each time Kubernetes creates such a pod, the KVA (Kubernetes Visibility Agent) Informer notifies ServiceNow to create a new pod CI, and the label is copied to `cmdb_key_value` on that pod CI. Tag-based Service Mapping then matches the tag value against the population rule configured for the service instance and records the membership in the **`svc_ci_assoc`** table.

5.3. Client-side Model

The client-side scenario is of interest on its own as there are so many workstation and mobile client applications. In the client-side scenario the log message occurs on workstation or mobile device that is not managed in the CMdb. As ServiceNow does not manage the client device in the CMdb, there are no CIs to attach to any log alert or incident. One way to address this is to create a synthetic CI that represents the application running on any client device. For this we choose the Application Service class(cmdb_ci_service_discovered) to instantiate a CI for the Spark Client. This CI will not have any infrastructure CIs associated with it as the client device is not managed in the CMdb.

Figure 7 illustrates the Client-side model of the Spark Client. It shows a device (and host) independent model consisting of a single Application Service CI that represents the Spark Client.

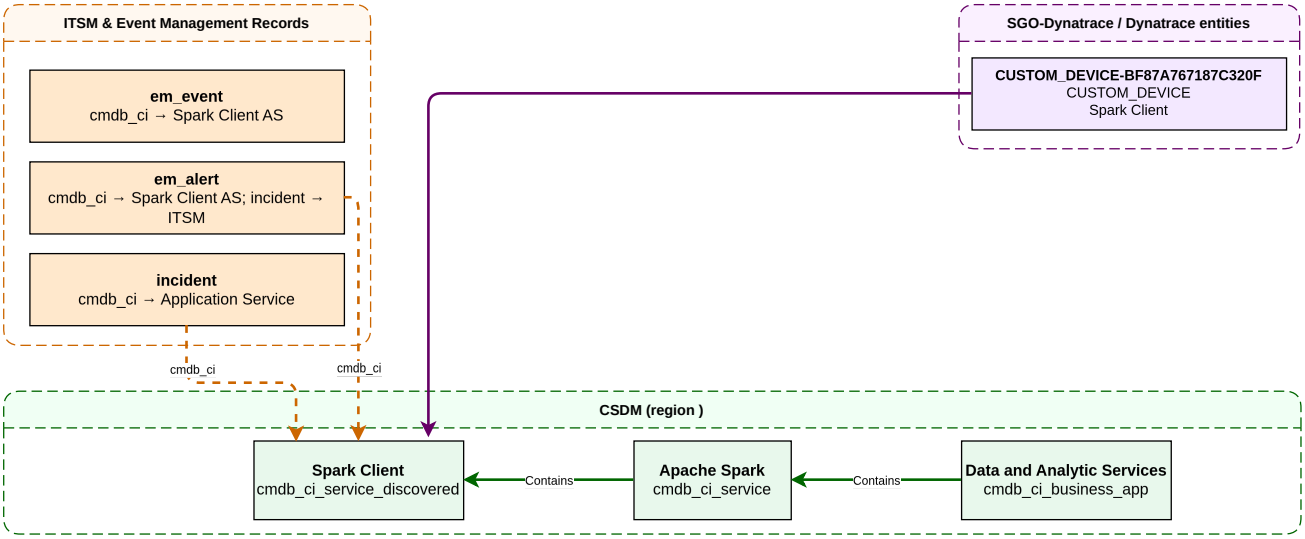

Figure 7. Spark Client-Side Model

The application service YAML specification for the Spark Client is shown below.

Listing 3. Spark CSDM Model for the Spark Client

```
- name: Spark Client
  identifier: spark-client
  platform: host
  service_mapping: manual
  discover: false
```

```
    depends_on:
      - Spark Master
```

Now, given the Spark Client application service how do we map a log message on the client device to the Spark Client application service? This is done by creating a custom device entity in Dynatrace that represents the Spark Client. This custom device entity must then be included in any log problems sent as events to ServiceNow from Dynatrace's OneClient agent on the device to Dynatrace's OpenPipeline.

5.4. Kubernetes Pod CIs

In this section we drill down into the details of how Kubernetes Pod CIs are created and how they are correlated to the application service. The pod tags are labels that are set on the pod manifest. These labels are then used to create the Kubernetes Pod CIs. Pod manifest labels are the single source of truth for tag-based Service Mapping on Spark workloads.

Tag-based Service Mapping matches workload CIs to a service instance using `cmdb_key_value` rows copied from these pod labels. The ServiceNow label uses the vendor-domain prefix `servicenow.com/`. There is no hard standard for the key name; this implementation uses the single key listed in the table below.

Table 3. ServiceNow pod label keys for tag-based Service Mapping

Key	Role
`servicenow.com/service-instance`	The only tag filter. Value equals the service instance **display name** (for example `Spark-Master` in the Kubernetes label character set, or `Elasticsearch` verbatim on Docker). Tag-based Service Mapping uses this to record matching workloads as members of that service instance in `svc_ci_assoc`.

The `servicenow.com/service-instance` tag is required on every workload that belongs to a tag-mapped service instance; no other ServiceNow tags (environment, location) are used. The tag value must match the display name of the service instance CI (spaces written as hyphens for Kubernetes labels). This is a naming contract that the teams need to keep in mind: renaming a service instance requires updating the workload labels and the tag population rule together.

Listing 4 illustrates the pod template for the Spark Worker.

Listing 4. Spark Worker Pod Template

```
apiVersion: v1
kind: Pod
metadata:
  name: spark-worker-lab1-0
  labels:
    servicenow.com/service-instance: Spark-Worker
    app.kubernetes.io/name: spark-worker
spec:
  containers:
    - name: spark-worker
      env:
        - name: POD_NAME
          valueFrom:
```

```
            fieldRef:
              fieldPath: metadata.name
      volumeMounts:
        - name: app-logs
          mountPath: /opt/spark/logs
          subPathExpr: $(POD_NAME)
  volumes:
    - name: app-logs
      persistentVolumeClaim:
        claimName: spark-logs-pvc
```

5.5. Tag-Based Service Mapping and svc_ci_assoc

Getting a pod (or container) associated with its service instance is a meeting of two halves that are configured independently and joined by ServiceNow:

1. **The workload side.** The deployment manifests carry the `servicenow.com/service-instance` label (single source of truth). KVA (Kubernetes) or the Docker discovery sync copies the label into `cmdb_key_value` on the workload CI. This happens continuously as workloads come and go — no per-pod automation is involved.
2. **The service side.** When the CSDM specification is deployed (`csdm/deploy.yml`), every service instance with `service_mapping: tags` gets a **tag population rule**: the deploy calls the Service Mapping Operations REST API (`/populate_tags`), which invokes `SMServiceByTagsUtils.updateServiceFromTagsList()` against the service instance sys_id with the single tag predicate `servicenow.com/service-instance = <display name>`.

ServiceNow's tag-based Service Mapping engine then evaluates the population rule against `cmdb_key_value` and records each matching workload CI as a member of the service instance in the **`svc_ci_assoc`** table (Service CI Association). This membership — not a CMDB relationship — is what incident automation traverses.

Two design rules follow from this model:

- **No ownership edges in `cmdb_rel_ci`.** Runtime topology relationships (pod → node → host, workload → pod) are imported by the Service Graph Connector and remain Dynatrace's Smartscape view. Service-instance ownership lives only in `svc_ci_assoc`, maintained by tag-based Service Mapping. No playbook or script materializes `Contains::Contained by` edges between service instances and workloads.
- **Elasticity without automation in the critical path.** Once the service instance and its population rule exist, new replicas of a known workload (for example a scaled-out Spark Worker Deployment) are picked up by KVA and joined to the service instance by Service Mapping alone. External automation (Ansible today, a Python utility in the future) is needed only when an **entirely new** service instance is introduced — a controlled CSDM onboarding step, not a runtime dependency.

5.6. ServiceNow — SGC Topology Import

Table 4. SGC topology import — Dynatrace entity to CMDB class

SGC scheduled import	Dynatrace entity type	CMDB class	Role in log incidents
Hosts	HOST	cmdb_ci_linux_server	Fallback when log tails on host and pod entity unavailable
Kubernetes Pod	CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod	Preferred alert/event CI for pod-scoped logs
Process Groups	PROCESS_GROUP	cmdb_ci_appl	Alternative when Davis binds to JVM process
Application Relationships	Smartscape edges	cmdb_rel_ci	Optional topology context

Each imported entity receives a **sys_object_source** row: name=SGO-Dynatrace, id=<Dynatrace entityId>, target_sys_id → CMDB CI sys_id.

Configure on the ServiceNow instance:

- Service Graph Connector for Observability – Dynatrace (scoped app 'sn_dynatrace_integ')
- Inbound event listener: POST /api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace
- Scheduled imports for hosts, Kubernetes pods, and process groups covering the cluster namespace

5.7. Dynatrace — management zone

Place hosts, Kubernetes cluster entities, and pods that run the application in a **management zone** dedicated to the integration (for example **Application Observability**). The **alerting profile** for ServiceNow problem notifications must scope to that management zone so only in-scope problems are forwarded.

Chapter 6. Chapter 5 — Client Mode Use Case

This chapter walks the Log to Incident reference path for **Spark Client Mode** only: application logs under `/mnt/spark/client-logs/`, OneAgent custom log source file tail, OpenPipeline Client enrichment (`spark.as_identifier`), and the Client Davis processor. Cluster Mode (container stdout) is Chapter 6.

6.1. Phase 2 — Spark Application

Now that we have defined the CMdb model for the Spark application, we can walk through the process and configurations needed to emit a Spark Client Mode log line with requisite metadata.

To start with, Listing 5 illustrates a sample log file from a Client Mode Spark Driver.

Listing 5. Spark Log File Sample

```
2026-07-20 12:40:19 WARN  Utils:250 - Your hostname, Lab3, resolves to a loopback address: 127.0.1.1; using
192.168.1.207 instead (on interface enp3s0)
2026-07-20 12:40:19 WARN  Utils:244 - Set SPARK_LOCAL_IP if you need to bind to another address
2026-07-20 12:40:27 WARN  TaskSetManager:250 - Lost task 0.0 in stage 0.0 (TID 0) (10.244.2.145 executor 0):
org.apache.spark.SparkException: [FAILED_READ_FILE.FILE_NOT_EXIS
T] Encountered error while reading file file:///mnt/spark/data/elements/Periodic_Table_Of_Elements.csv. File does not
exist. It is possible the underlying files have been up
dated.
2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job
```

As typical with many applications, there is not a lot of structure to the log files. There is the ever present time stamp, an error level, the file and line number of the log message, and the message itself. Of particular importance is the log level. Instead of matching a long list of critical log message patterns, we can use it to decide if the log message should be considered a critical issue. For the reference implementation we choose to use both the WARN and ERROR levels. We use warnings for convenience as they are easier to generate.

There are several key questions we need to answer:

- Where will we keep the log files?
- What key metadata will we need to extract from the log files?
- What other key metadata will we need about the log files?

6.1.1. Log File Directory Structure

Client Mode uses a **directory-based** Log4j RollingFile under `/mnt/spark/client-logs/<instance>/` (node-local bind mount — not NFS). OneAgent tails those files via a custom log source. The **instance** path element defaults to the wrapper PID of the Spark Driver; parallel drivers on one host set **SPARK_DRIVER_INSTANCE**. OpenPipeline sets `spark.mode=Client`, composes `spark.instance=<host.name>:<instance>`, and stamps `spark.as_identifier=spark-client`.

Listing 6 illustrates the client-mode log directories.

Listing 6. Spark client-mode log directories

```
/mnt/spark/client-logs/                 # Client mode (node-local)
 ├──── 12345/                             # default: wrapper PID
 ├──── par-a/                             # SPARK_DRIVER_INSTANCE override
 └──── par-b/
```

6.1.2. Log4j configuration

Client Mode (`spark/conf/log4j2-client.properties`) uses Console plus RollingFile. `run-chapters.sh` sets `SPARK_LOG_DIR=/mnt/spark/client-logs/<instance>/` and `-Dlog4j2.configurationFile=⋯/log4j2-client.properties`. OneAgent harvests the RollingFile tree via the custom log source below.

Listing 7. Log4j RollingFile — client-side

```
appender.rolling.type = RollingFile
appender.rolling.name = RollingFile
appender.rolling.fileName = ${env:SPARK_LOG_DIR:-/mnt/spark/client-logs/default}/spark-app.log
appender.rolling.filePattern = ${env:SPARK_LOG_DIR:-/mnt/spark/client-logs/default}/spark-app-%d{yyyy-MM-dd}-%i.log
appender.rolling.filePermissions = rw-r--r--
appender.rolling.layout.type = PatternLayout
appender.rolling.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n%ex
appender.rolling.policies.type = Policies
appender.rolling.policies.time.type = TimeBasedTriggeringPolicy
appender.rolling.policies.time.interval = 1
appender.rolling.policies.size.type = SizeBasedTriggeringPolicy
appender.rolling.policies.size.size = 20MB
appender.rolling.strategy.type = DefaultRolloverStrategy
appender.rolling.strategy.max = 10
```

6.2. Phase 2 — OneAgent

OneAgent is Dynatrace's single, unified data-collection agent that runs on each host. It performs full-stack monitoring of hosts, containers, and applications. In the Client Mode Log to Incident path it tails files under `/mnt/spark/client-logs/` and sends each entry with contextual metadata to Dynatrace.

We need to tell OneAgent where to find the Client Mode log files and what contextual metadata to include with each log entry.

6.2.1. Log Metadata

The log metadata is the key metadata fields that we need to extract from the log files or the log file context. This depends on what fields are required, the available log file context, the available fields in a log line, Dynatrace's ability to parse the log file context, and the OpenPipeline's ability to parse the log file context.

The key requirements for Client Mode are that: - We need to know the log level in order to determine which issues are critical or not. - We need to mark the stream as Client Mode and capture the driver **instance** differentiator. - We need an Application Service bind key (`spark.as_identifier`) for ServiceNow.

OneAgent provides a limited set of default metadata fields (raw line, log level, log source path). The custom log source below stamps the Client Mode `spark.*` attributes from the path pattern.

Listing 8. Custom Log Source Configuration (Client Mode paths only)

```
{
  "schemaId": "builtin:logmonitoring.custom-log-source-settings",
  "scope": "environment",
  "value": {
    "enabled": true,
    "config-item-title": "{{ dt_sn_spark_log_custom_source_name }}",
    "custom-log-source": {
      "type": "LOG_PATH_PATTERN",
      "values-and-enrichment": [
        {
          "path": "/mnt/spark/client-logs/*/spark-app.log",
          "enrichment": [
            { "type": "attribute", "key": "spark.log.path", "value": "${0}" },
            { "type": "attribute", "key": "spark.mode", "value": "Client" },
            { "type": "attribute", "key": "spark.driver.instance", "value": "${1}" },
            { "type": "attribute", "key": "spark.as_identifier", "value": "spark-client" }
          ]
        },
        {
          "path": "/mnt/spark/client-logs/*/spark-app-*.log",
          "enrichment": [
            { "type": "attribute", "key": "spark.log.path", "value": "${0}" },
            { "type": "attribute", "key": "spark.mode", "value": "Client" },
            { "type": "attribute", "key": "spark.driver.instance", "value": "${1}" },
            { "type": "attribute", "key": "spark.as_identifier", "value": "spark-client" }
          ]
        }
      ]
    }
  }
}
```

Given the above template, OneAgent will extract the following metadata fields and send them to Dynatrace.

Table 5. OneAgent and custom log source metadata fields (Client Mode)

Attribute	How it is set	Value
`content`	OneAgent (raw line)	Full Log4j line
`loglevel`	OneAgent (severity from line text)	`WARN`, `ERROR`, `INFO`, …
`log.source`	OneAgent	Registered path under `/mnt/spark/client-logs/…`
`spark.log.path`	Custom log source `${0}`	Literal file path on the node
`spark.mode`	Custom log source	`Client`
`spark.driver.instance`	Custom log source `${1}`	Driver directory under `/mnt/spark/client-logs/` (`par-a`, PID, …)
`spark.as_identifier`	Custom log source	`spark-client` (Application Service bind key for ServiceNow)

6.3. Phase 3 — Dynatrace

Once the logs are sent to Dynatrace, they must be routed to an OpenPipeline that processes the log entry. During the pipeline some log entries may be further processed then dropped, converted to problems, or stored in the Grail bucket. Those events converted to a problem are then forwarded to ServiceNow.

6.3.1. Log Routing

To route the logs to the designated OpenPipeline to process Spark log entries. , we need to configure the Logs Routing object. This object is used to route the logs to the OpenPipeline for Spark logs.

Listing 9. Spark OpenPipeline logs routing

```
{
  "schemaId": "builtin:openpipeline.logs.routing",
  "scope": "environment",
  "value": {
    "routingEntries": [
      {
        "enabled": true,
        "pipelineType": "custom",
        "pipelineId": "vu9U3hXa3q0AAAABA...zIxOb7vVN4V2t6t",
        "matcher": "matchesValue(log.source, \"/mnt/spark/client-logs/*\") OR matchesValue(log.source, \"Container
Output\")",
        "description": "Spark Lab log alerts (ERROR/WARN)"
      }
    ]
  }
}
```

Where:

- `pipelineId` is the ID of the Spark log processing OpenPipeline

This rule routes Client file tails (`/mnt/spark/client-logs/*`) and Cluster Container Output (see Chapter 6) to the shared Spark log processing OpenPipeline. This chapter covers the Client path only.

6.3.2. OpenPipeline processing

The Spark log processing OpenPipeline has three stages of processing:

1. **Processors** - where the log entries are processed and the metadata fields are extracted or defined
2. **Davis** - where the log entries are matched to a Davis processor to generate a Davis event that is ultimately convertert to a problem and forwarded to ServiceNow.
3. **Grail** - where the Davis events are stored in the Grail bucket.

The template for the Spark OpenPipeline is shown below. Notice the last three stages are placeholders. We will define them as we touch on each processing stage

Listing 10. Spark OpenPipeline Configuration

```
{
  "schemaId": "builtin:openpipeline.logs.pipelines",
  "scope": "environment",
  "value": {
    "displayName": "Spark Lab - log alerts",
    "customId": "spark-lab-log-alerts",
    "routing": "routable",
    "groupRole": "memberPipeline",
    "processing": {},
    "davis": {},
    "storage": {}
  }
}
```

6.3.2.1. Processors

The OneAgent custom log source in Listing 8 already sets `spark.*` variables prior to entry of the pipeline. In addition to the standard Dynatrace fields like `content`, `loglevel`, and `log.source`, the available fields on entry to the pipeline processors are:

Table 6. OpenPipeline entry fields (Client Mode)

Field Name	Client Mode
spark.mode	`Client` (from custom source)
spark.driver.instance	path segment under `/mnt/spark/client-logs/`
spark.instance	`<host.name>:<driver-instance>` (composed in pipeline)
spark.as_identifier	`spark-client` (from custom source / pipeline; SN AS bind)
dt.source_entity	HOST (OneAgent file tail; in MZ). SN binds via `spark.as_identifier`, not HOST

The Client-relevant OpenPipeline processors are defined in Listing 11, below:

1. **`tag-pipeline-custom-id`** — stamps the OpenPipeline customId in a `pipeline` field on every log record for Grail filtering.
2. **`compose-client-instance`** — defines `spark.instance` from `host.name` and `spark.driver.instance`.
3. **`ensure-client-as-identifier`** — sets `spark.as_identifier=spark-client` when missing so ServiceNow can bind the Application Service.
4. **`extract-log4j-class-line`** — parses the Log4j short class and line (`Class:Line`) from the `content` field.

(Cluster Mode adds `enrich-cluster-from-k8s` on Container Output; see Chapter 6.)

Listing 11. Spark OpenPipeline processing processors

```
{
    "processing": {
      "processors": [
        {
          "id": "tag-pipeline-custom-id",
          "type": "fieldsAdd",
          "enabled": true,
```

```
          "description": "Convention: stamp OpenPipeline customId on every log record for easy Grail filtering",
          "matcher": "true",
          "fieldsAdd": {
            "fields": [
              { "name": "pipeline", "value": "{{ dt_sn_spark_openpipeline_pipeline_custom_id | mandatory }}" }
            ]
          }
        },
        {
          "id": "compose-client-instance",
          "type": "dql",
          "enabled": true,
          "description": "Client: compose spark.instance from host.name + spark.driver.instance",
          "matcher": "spark.mode == \"Client\" AND isNotNull(spark.driver.instance)",
          "dql": {
            "script": "fieldsAdd spark.instance = concat(toString(coalesce(host.name, \"unknown\")), \":\",
toString(spark.driver.instance))"
          }
        },
        {
          "id": "ensure-client-as-identifier",
          "type": "fieldsAdd",
          "enabled": true,
          "description": "Client: Application Service identifier for ServiceNow bind (dt.source_entity left as
OneAgent HOST)",
          "matcher": "spark.mode == \"Client\" AND isNull(spark.as_identifier)",
          "fieldsAdd": {
            "fields": [
              { "name": "spark.as_identifier", "value": "spark-client" }
            ]
          }
        },
        {
          "id": "extract-log4j-class-line",
          "type": "dql",
          "enabled": true,
          "description": "Parse Log4j short class and line (Class:Line) on all Spark lines for Grail and Davis
unique_identifier",
          "matcher": "true",
          "dql": {
            "script": "parse content, \"TIMESTAMP('yyyy-MM-dd HH:mm:ss') SPACE{1,} WORD:spark.log.level SPACE{1,}
LD:spark.log.class ':' INT:spark.log.line LD\""
          }
        }
      ]
    }
}
```

`spark.as_identifier` carries the Application Service `identifier` (`spark-client`) that ServiceNow uses to bind the Client Application Service; Davis leaves `dt.source_entity` as the OneAgent HOST.

The Java class and line number uniquely identify the source of a log entry. For log entries that represent error conditions the class and line number form part of an error signature that we can use to identify errors that likely have the same root cause. Combined with mode information, we use the class name and line number (class:line)as the unique identifier for the log entry. This is used to correlate the log entry to a ServiceNow incident.

6.3.2.2. Davis

Davis processing enables communication of critical issues to ServiceNow and provides opportunities to reduce noise or increase the strength of the signals sent to ServiceNow. Noise is reduced by eliding potential events that match an existing Davis event. Signal strength is

increased by creating a Davis problem and grouping events that have the same root cause into a single Davis problem. We discuss how Davis processors work and then how we define the processors for handling Spark log entries.

6.3.2.2.1. Davis Processing

The Davis processors create Davis events or if a pipeline candidate's identity matches an existing Davis event, it will update the existing event. Davis defines identity as the tuple:

```
(event.name, event.type, dt.source_entity)
```

If this tuple is the same across the candidate in the pipeline and an existing Davis event, the existing event will be updated. If there was no match, a new Davis event will be created.

Once a Davis event is created, the Davis processor may create or update a Davis problem. A Davis processor is a rule that has a matcher. If the condition of the matcher is met, the Davis processor will create a Davis problem for an event or group the event into an existing Davis problem. If the condition of the matcher is not met, the Davis processor will drop the event from any further Davis processing, but not from any other Pipeline processing, like persisting the event to Grail.

If the matcher condition is met, the Davis processor will create a Davis problem or group the event into an existing Davis problem. Grouping is dependent on the problem's:

1. the matcher condition is met
2. `dt.davis.is_merging_allowed` property is set to `true`
3. the event's timestamp being with the 'dt.davis.event_timeout' window of the existing Davis problem
4. the event's `dt.source_entity` being the same as the existing problem's entity or topologically close in Smartscape as in process to host relationships or closely related services

Once the metadata fields are defined in the processors, we are ready to create Davis problems for those "critical" Client Mode log entries with a log level of WARN or ERROR. This chapter documents the **Client** Davis processor plus a fail-visible processor when `spark.mode` is missing on a Client file path. The Cluster Davis processor is in Chapter 6.

6.3.2.2.2. Davis Configuration for Spark

You should consult application development and operations teams on analysis of log message severity and root cause equivalence classes of log messages. For the purpose of this reference implementation, we need to make some assumptions. We define a unique error signature as being the class name and line number of a critical log entry. For now, we assume a critical error is log message with a severity of WARN or ERROR. We use WARNs merely for convenience as they are more likely to be generated.

Furthermore, we assume that the same error signature across pods and/or client instances does **not** necessarily have the same root cause. For example, in a enterprise getting the same error around the same time across clusters in different regions likely does not have the same root cause. Granted, having the same error signature across pods in the same region likely does have

the same root cause. Also, we assume that different error signatures within the same pod or client do no necessarily have the same root cause. However, Davis **does not** have the fidelity to define event by root cause and differentiate between different remote or local clusters or clients.

Since different critical errors may likely have different root causes, we want to create a Davis problem (and ServiceNow incident) for each different critical error. To arrange this we need to ensure that the tuple `(event.name, event.type, dt.source_entity)` is unique for each critical error. Tuple uniqueness will result in a new Davis problem being created for each critical error. The main component over which we have some control is `event.name`. To ensure uniqueness for different error signatures, we will define `event.name` in terms of the error's (log entries) class name and line number.

We will define `event.name` as:

```
Critical log.level error at {spark.log.class}:{spark.log.line}
```

The Davis processors would group events, i.e. different error signatures, into a single Davis problem. At least for now we want to avoid this. To do so requires setting the `dt.davis.is_merging_allowed` property to `false`. The net results is that we will only get one Davis problem for each unique error signature across all pods or clients. This may not be ideal for different dispersed clusters, but probably the best we can do at this time.

We could put `{spark.mode}` in the `event.name` to group errors by mode, but if the error signature was the same across modes, we would still want to have a single Davis event as the root cause would likely be the same.

The Davis process define the following properties across all three processors:

Table 7. Davis Properties (Client Mode)

Property Name	Description
`event.type`	`ERROR_EVENT` (Dynatrace Settings API fixed enum)
`spark.event_kind`	`CRITICAL_LOG_EVENT` (lab semantic label; ServiceNow remaps `em_event.type` / `em_alert.type`)
`event.name`	`Spark critical {loglevel} error at {spark.log.class}:{spark.log.line}`
`event.description`	includes `{spark.log.class}:{spark.log.line}`, path, and `spark.as_identifier=` for SN bind
`dt.source_entity`	HOST (OneAgent file tail; in MZ). SN binds via `spark.as_identifier`, not HOST
`spark.as_identifier`	`spark-client` (stamped into description + properties)
`event.unique_identifier`	`Spark Client Critical {loglevel} error {spark.log.class}:{spark.log.line}`
`pipeline`	`spark-lab-log-alerts` (OpenPipeline customId; stamped in processing)
`dt.davis.is_merging_allowed`	`false` to prevent merging of multiple Davis events into the same problem
`dt.davis.event_timeout`	`15m` (unused while merging is disabled)

The Davis processors are defined in Listing 12, below.

Listing 12. Spark OpenPipeline Davis processors

```
{
    "davis": {
      "processors": [
        {
          "id": "spark-client-warn-error-davis",
          "type": "davis",
          "enabled": true,
          "description": "Client WARN/ERROR → ERROR_EVENT (DT enum); spark.event_kind=CRITICAL_LOG_EVENT; stamp
spark.as_identifier; leave HOST as dt.source_entity",
          "matcher": "(loglevel == \"WARN\" OR loglevel == \"ERROR\") AND spark.mode == \"Client\" AND
isNotNull(spark.log.class) AND isNotNull(spark.log.line)",
          "davis": {
            "properties": [
              { "key": "event.type", "value": "ERROR_EVENT" },
              { "key": "spark.event_kind", "value": "CRITICAL_LOG_EVENT" },
              { "key": "event.name", "value": "Spark critical {loglevel} error at {spark.log.class}:{spark.log.line}"
},
              {
                "key": "event.description",
                "value": "Spark Client critical {loglevel} error at {spark.log.class}:{spark.log.line} in
{spark.log.path} spark.event_kind=CRITICAL_LOG_EVENT spark.as_identifier={spark.as_identifier}: {content}"
              },
              { "key": "dt.davis.is_merging_allowed", "value": "false" },
              { "key": "spark.as_identifier", "value": "{spark.as_identifier}" },
              { "key": "event.unique_identifier", "value": "Spark Client Critical {loglevel} error
{spark.log.class}:{spark.log.line}" },
              { "key": "pipeline", "value": "{pipeline}" },
              { "key": "dt.davis.is_entity_remapping_allowed", "value": "false" },
              { "key": "dt.davis.event_timeout", "value": "15m" }
            ]
          }
        },
        {
          "id": "spark-enrichment-missing-davis",
          "type": "davis",
          "enabled": true,
          "description": "Fail-visible: WARN/ERROR without spark.mode (Client file path)",
          "matcher": "(loglevel == \"WARN\" OR loglevel == \"ERROR\") AND isNull(spark.mode) AND
matchesValue(log.source, \"/mnt/spark/client-logs/*\")",
          "davis": {
            "properties": [
              { "key": "event.type", "value": "ERROR_EVENT" },
              { "key": "spark.event_kind", "value": "CRITICAL_LOG_EVENT" },
              { "key": "event.name", "value": "Spark log missing spark.mode enrichment on {log.source}" },
              {
                "key": "event.description",
                "value": "Log enrichment did not set spark.mode for {log.source} spark.event_kind=CRITICAL_LOG_EVENT:
{content}"
              },
              { "key": "event.unique_identifier", "value": "enrichment-missing|{log.source}" },
              { "key": "pipeline", "value": "{pipeline}" },
              { "key": "dt.davis.is_merging_allowed", "value": "false" },
              { "key": "dt.davis.event_timeout", "value": "15m" }
            ]
          }
        }
      ]
    }
}
```

Once a Davis problem is created the problem is forwarded to ServiceNow via the alerting profile

in Listing 14.

6.3.2.3. Storage

After the problems are created, Dynatrace stores all of the **log records** in the default_logs bucket (storage processors below).

Listing 13. Spark OpenPipeline storage (default_logs bucket)

```
{
    "storage": {
      "processors": [
        {
          "id": "default-storage",
          "type": "bucketAssignment",
          "enabled": true,
          "description": "Store in default logs bucket",
          "matcher": "true",
          "bucketAssignment": { "bucketName": "default_logs" }
        }
      ]
    }
}
```

6.3.3. Alerting

To emit a Davis problem to ServiceNow we have to create an alerting profile and associate it with the webhook endpoint in ServiceNow designed to receive the problem from Dynatrace. We also have to specify the webhook payload format.

6.3.3.1. Alerting Profile

The alerting profile couples a management zone to a set of severity level and event filters on the problems. Matching problems will be emitted to ServiceNow. THe full alerting profile template is shown below.

Listing 14. Alerting profile for Spark Observability to ServiceNow

```
{
  "schemaId": "builtin:alerting.profile",
  "scope": "environment",
  "value": {
    "name": "Application Observability - ServiceNow",
    "severityRules": [
      { "severityLevel": "AVAILABILITY", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "ERRORS", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "PERFORMANCE", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "RESOURCE_CONTENTION", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" },
      { "severityLevel": "CUSTOM_ALERT", "delayInMinutes": 0, "tagFilterIncludeMode": "NONE" }
    ],
    "eventFilters": [],
    "managementZone": "<Spark Observability - MANAGMENT ZONE OBJECT-ID>"
  }
}
```

Note that alerting profiles management zone is set to the Spark Observability management zone. This is important as the alerting profile will now filter out any problems that do not have an

`dt.source_entity` in the Spark Observability management zone. This means that the impacted entities (hosts for OneAgent file tails and CPU) must be in the Spark Observability management zone, too.

6.3.3.2. Webhook Endpoint

The webhook endpoint template couples the alerting profile to the webhook endpoint in ServiceNow designed to receive the problem from Dynatrace. The full webhook endpoint template is shown below.

Listing 15. Problem notification webhook endpoint (SGC)

```
enabled: true
type: WEBHOOK
displayName: "ServiceNow brooks-lab - Spark Observability"
alertingProfile: "<Application Observability - ServiceNow OBJECT-ID"
webHookNotification:
  url: "https://optimizincdemo1.service-now.com/api/sn_em_connector/em/inbound_event?source=SGO-Dynatrace
"
```

6.3.4. Problem notification payload template

The template in Problem notification payload specification is substituted by Dynatrace at send time. `ProblemDetailsJSON.rankedEvents[]` carries the OpenPipeline Davis properties from OpenPipeline Davis processors (`event.description`, entity ids, severity). Those ranked events are the **problem specification** that flows into ServiceNow `em_event.description` (via `ProblemDetailsText`) and into `em_event.additional_info`.

Listing 16. Specification 2.6.2-1. Problem notification payload (SGC)

```
{
  "ConnectionId": "<sgc-connection-sys-id>",
  "ProblemID": "{ProblemID}",
  "ProblemTitle": "{ProblemTitle}",
  "ProblemSeverity": "{ProblemSeverity}",
  "ProblemImpact": "{ProblemImpact}",
  "ProblemDetailsJSON": {ProblemDetailsJSON},
  "ProblemDetailsText": "{ProblemDetailsText}",
  "ImpactedEntities": {ImpactedEntities},
  "ImpactedEntity": "{ImpactedEntity}",
  "ProblemURL": "{ProblemURL}",
  "State": "{State}",
  "Tags": "{Tags}",
  "PID": "{PID}"
}
```

A complete example of the problem notification payload is shown below.

Listing 17. Example Client Mode problem notification payload

```
{
  "ConnectionId": "a1b2c3d4e5f678901234567890abcdef0",
  "ProblemID": "-9876543210987654321",
  "ProblemTitle": "Spark critical ERROR error at TaskSetManager:270",
  "ProblemSeverity": "ERROR",
  "ProblemImpact": "INFRASTRUCTURE",
  "ProblemDetailsJSON": {
    "displayName": "Spark critical ERROR error at TaskSetManager:270",
```

```
    "problemId": "-9876543210987654321",
    "rankedEvents": [
      {
        "entityId": "HOST-A1B2C3D4E5F67890",
        "entityName": "Lab3",
        "eventType": "ERROR_EVENT",
        "severityLevel": "ERROR",
        "title": "Spark critical ERROR error at TaskSetManager:270",
        "eventDescription": "Spark Client critical ERROR error at TaskSetManager:270 in /mnt/spark/client-logs/par-
a/spark-app.log spark.event_kind=CRITICAL_LOG_EVENT spark.as_identifier=spark-client: 2026-07-20 12:40:29 ERROR
TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job"
      }
    ],
    "startTime": 1719502426000,
    "endTime": -1
  },
  "ProblemDetailsText": "Spark Client critical ERROR error at TaskSetManager:270 in /mnt/spark/client-logs/par-
a/spark-app.log spark.event_kind=CRITICAL_LOG_EVENT spark.as_identifier=spark-client:\n2026-07-20 12:40:29 ERROR
TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job\n\n1 impacted entity: Lab3",
  "ImpactedEntities": [
    {
      "entityId": "HOST-A1B2C3D4E5F67890",
      "name": "Lab3",
      "type": "HOST"
    }
  ],
  "ImpactedEntity": "Lab3",
  "ProblemURL": "https://<tenant>.apps.dynatrace.com/ui/problems/<problem-id>",
  "State": "OPEN",
  "Tags": "Project:spark-observability",
  "PID": "-9876543210987654321"
}
```

Chapter 7. Chapter 6 — Cluster Mode Use Case

This chapter mirrors Chapter 5 for **Spark Cluster Mode**: Log4j2 **Console** → container stdout, DynaKube `logMonitoring: {}` → `log.source=Container Output`, OpenPipeline enrichment from `k8s.pod.name` (`spark.mode=Cluster`, `spark.pod_identifier`), and the Cluster Davis processor. There is **no** custom log source path on `/mnt/spark/logs` for application logs.

7.1. Phase 2 — Spark Application

Cluster Mode Spark components (Master, Workers, History Server, executors) emit application logs to **stdout** so the kubelet / OneAgent Log module can ingest them with Kubernetes entity context.

Listing 18 illustrates a sample Cluster Mode log line (same Log4j pattern as Client Mode).

Listing 18. Spark Cluster Mode log sample (stdout)

```
2026-07-20 12:40:29 ERROR TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job
```

Criticality for the reference implementation remains WARN and ERROR log levels.

7.1.1. Log emission (container stdout)

Cluster Mode does **not** rely on OneAgent file-tail under `/mnt/spark/logs/<pod>/` for Log-to-Incident application logs. Pods write Log4j application lines to the Console appender (container stdout). Dynatrace ingests those streams as `log.source=Container Output`.

A separate RollingFile under `/opt/spark/logs` may still exist for the OTel EventEmitter path; that file is **not** used for L2I Davis alerts.

7.1.2. Log4j configuration

Cluster Mode uses `spark/conf/log4j2-cluster.properties`: root logger → **Console** only for application logs.

Listing 19. Log4j Console — Cluster Mode (container stdout)

```
# ConsoleAppender — kubelet /var/log/pods → Dynatrace Container Output
appender.console.type = Console
appender.console.name = Console
appender.console.layout.type = PatternLayout
appender.console.layout.pattern = %d{yyyy-MM-dd HH:mm:ss} %-5p %c{1}:%L - %m%n%ex

# Root logger — Console only (container stdout)
rootLogger.level = info
rootLogger.appenderRefs = console
rootLogger.appenderRef.console.ref = Console
```

7.2. Phase 2 — OneAgent

OneAgent on each Kubernetes node collects container stdout/stderr when DynaKube enables the Log module. Unlike Client Mode, Cluster Mode application logs do **not** use a custom log source path pattern for `/mnt/spark/logs`.

7.2.1. Log Metadata

Listing 20. DynaKube logMonitoring (Container Output)

```
apiVersion: dynatrace.com/v1beta5
kind: DynaKube
metadata:
  name: dynakube
spec:
  # OneAgent Log module: autodiscover container stdout/stderr
  # (log.source=Container Output). Required for Cluster Mode Spark
  # app logs after log4j2-cluster Console appender.
  logMonitoring: {}
```

Repo: `observability/dynatrace/dynakube/dynakube.yaml.j2`.

Table 8. OneAgent Container Output metadata fields (Cluster Mode)

Attribute	How it is set	Value
`content`	OneAgent (raw line)	Full Log4j line from stdout
`loglevel`	OneAgent (severity from line text)	`WARN`, `ERROR`, `INFO`, …
`log.source`	OneAgent Log module	`Container Output`
`k8s.namespace.name`	OneAgent / Kubernetes enrichment	e.g. `spark`
`k8s.pod.name`	OneAgent / Kubernetes enrichment	e.g. `spark-master-0` (pod `metadata.name`)
`spark.mode` / `spark.pod_identifier`	OpenPipeline (not custom log source)	Set in processing from `k8s.pod.name` (below)
Entity context	OneAgent Log module	May attach CAI / PGI / process context — unlike Client Mode HOST file-tail

7.3. Phase 3 — Dynatrace

7.3.1. Log Routing

Cluster Container Output shares the Spark Lab OpenPipeline with Client file tails (same routing object as Chapter 5 routing). Matcher includes `log.source` = `Container Output`.

Listing 21. Spark OpenPipeline logs routing (Cluster-relevant matcher)

```
{
  "schemaId": "builtin:openpipeline.logs.routing",
  "scope": "environment",
  "value": {
    "routingEntries": [
      {
```

```
        "enabled": true,
        "pipelineType": "custom",
        "pipelineId": "vu9U3hXa3q0AAAABA...zIxOb7vVN4V2t6t",
        "matcher": "matchesValue(log.source, \"/mnt/spark/client-logs/*\") OR matchesValue(log.source, \"Container
Output\")",
        "description": "Spark Lab log alerts (Client file + Cluster Container Output)"
      }
    ]
  }
}
```

7.3.2. OpenPipeline processing

Same pipeline shell as Chapter 5 (`spark-lab-log-alerts`). Cluster Mode adds the `enrich-cluster-from-k8s` processor before Log4j parse / Davis.

Table 9. OpenPipeline entry fields (Cluster Mode)

Field Name	Cluster Mode
log.source	`Container Output`
k8s.pod.name	Kubernetes pod name (e.g. `spark-master-0`)
spark.mode	`Cluster` (from `enrich-cluster-from-k8s`)
spark.pod_identifier	`toString(k8s.pod.name)` (from pipeline; SN bind key)
spark.as_identifier	unset
dt.source_entity	OneAgent Log module entity context (CAI/PGI when associated). SN binds via `spark.pod_identifier`, not HOST file-tail

7.3.2.1. Processors

Listing 22. Spark OpenPipeline processing processors (Cluster Mode)

```
{
    "processing": {
      "processors": [
        {
          "id": "tag-pipeline-custom-id",
          "type": "fieldsAdd",
          "enabled": true,
          "description": "Convention: stamp OpenPipeline customId on every log record for easy Grail filtering",
          "matcher": "true",
          "fieldsAdd": {
            "fields": [
              { "name": "pipeline", "value": "{{ dt_sn_spark_openpipeline_pipeline_custom_id | mandatory }}" }
            ]
          }
        },
        {
          "id": "enrich-cluster-from-k8s",
          "type": "dql",
          "enabled": true,
          "description": "Cluster: Container Output → spark.mode + spark.pod_identifier from k8s.pod.name",
          "matcher": "matchesValue(log.source, \"Container Output\") AND matchesValue(k8s.namespace.name, \"spark\")
AND matchesValue(k8s.pod.name, \"spark-*\") AND isNull(spark.mode)",
          "dql": {
            "script": "fieldsAdd spark.mode = \"Cluster\", spark.pod_identifier = toString(k8s.pod.name)"
          }
        },
```

```
        {
          "id": "extract-log4j-class-line",
          "type": "dql",
          "enabled": true,
          "description": "Parse Log4j short class and line (Class:Line) for Grail and Davis unique_identifier",
          "matcher": "true",
          "dql": {
            "script": "parse content, \"TIMESTAMP('yyyy-MM-dd HH:mm:ss') SPACE{1,} WORD:spark.log.level SPACE{1,}
LD:spark.log.class ':' INT:spark.log.line LD\""
          }
        }
      ]
    }
}
```

`spark.pod_identifier` carries the Kubernetes pod name for the ServiceNow `svc_ci_assoc` membership lookup to the Application Service. OpenPipeline does **not** bake CAI entity ids at apply time.

7.3.2.2. Davis

Table 10. Davis Properties (Cluster Mode)

Property Name	Description
`event.type`	`ERROR_EVENT` (Dynatrace Settings API fixed enum)
`spark.event_kind`	`CRITICAL_LOG_EVENT` (ServiceNow remaps `em_event.type` / `em_alert.type`)
`event.name`	`Spark critical {loglevel} error at {spark.log.class}:{spark.log.line}`
`event.description`	includes class:line, `spark.pod_identifier=`, and `{content}` for SN bind
`dt.source_entity`	Log-module entity context when present (CAI/PGI). SN prefers `spark.pod_identifier` over HOST
`spark.pod_identifier`	Pod name (stamped into description + properties)
`event.unique_identifier`	`Spark Cluster Critical {loglevel} error {spark.log.class}:{spark.log.line}`
`pipeline`	`spark-lab-log-alerts`
`dt.davis.is_merging_allowed`	`false`
`dt.davis.event_timeout`	`15m`

Listing 23. Spark OpenPipeline Davis processors (Cluster Mode)

```
{
    "davis": {
      "processors": [
        {
          "id": "spark-cluster-warn-error-davis",
          "type": "davis",
          "enabled": true,
          "description": "Cluster WARN/ERROR from Container Output; stamp spark.pod_identifier for SN",
          "matcher": "(loglevel == \"WARN\" OR loglevel == \"ERROR\") AND spark.mode == \"Cluster\" AND
isNotNull(spark.log.class) AND isNotNull(spark.log.line) AND isNotNull(spark.pod_identifier)",
          "davis": {
            "properties": [
              { "key": "event.type", "value": "ERROR_EVENT" },
```

```
              { "key": "spark.event_kind", "value": "CRITICAL_LOG_EVENT" },
              { "key": "event.name", "value": "Spark critical {loglevel} error at {spark.log.class}:{spark.log.line}"
},
              {
                "key": "event.description",
                "value": "Spark Cluster critical {loglevel} error at {spark.log.class}:{spark.log.line} on
{spark.pod_identifier} spark.event_kind=CRITICAL_LOG_EVENT spark.pod_identifier={spark.pod_identifier}: {content}"
              },
              { "key": "dt.davis.is_merging_allowed", "value": "false" },
              { "key": "spark.pod_identifier", "value": "{spark.pod_identifier}" },
              { "key": "event.unique_identifier", "value": "Spark Cluster Critical {loglevel} error
{spark.log.class}:{spark.log.line}" },
              { "key": "pipeline", "value": "{pipeline}" },
              { "key": "dt.davis.is_entity_remapping_allowed", "value": "false" },
              { "key": "dt.davis.event_timeout", "value": "15m" }
            ]
          }
        },
        {
          "id": "spark-enrichment-missing-davis",
          "type": "davis",
          "enabled": true,
          "description": "Fail-visible: WARN/ERROR without spark.mode on Container Output",
          "matcher": "(loglevel == \"WARN\" OR loglevel == \"ERROR\") AND isNull(spark.mode) AND
matchesValue(log.source, \"Container Output\")",
          "davis": {
            "properties": [
              { "key": "event.type", "value": "ERROR_EVENT" },
              { "key": "spark.event_kind", "value": "CRITICAL_LOG_EVENT" },
              { "key": "event.name", "value": "Spark log missing spark.mode enrichment on {log.source}" },
              {
                "key": "event.description",
                "value": "Log enrichment did not set spark.mode for {log.source} spark.event_kind=CRITICAL_LOG_EVENT:
{content}"
              },
              { "key": "event.unique_identifier", "value": "enrichment-missing|{log.source}" },
              { "key": "pipeline", "value": "{pipeline}" },
              { "key": "dt.davis.is_merging_allowed", "value": "false" },
              { "key": "dt.davis.event_timeout", "value": "15m" }
            ]
          }
        }
      ]
    }
}
```

Once a Davis problem is created it is forwarded to ServiceNow via the same alerting profile / webhook as Chapter 5 (Listing 14).

7.3.2.3. Storage

Storage is the same default_logs bucket assignment as Chapter 5 (Listing 13).

7.3.3. Alerting

Alerting profile and brooks-lab webhook are shared with Client Mode. Ensure Kubernetes workloads (`CLOUD_APPLICATION spark-*`) and related CAI instances are in MZ **Spark Observability** so Container Output problems notify SGO.

7.3.4. Problem notification payload template

Payload shape matches Chapter 5 payload specification. Cluster Mode example:

Listing 24. Example Cluster Mode problem notification payload

```
{
  "ConnectionId": "a1b2c3d4e5f678901234567890abcdef0",
  "ProblemID": "-9876543210987654321",
  "ProblemTitle": "Spark critical ERROR error at TaskSetManager:270",
  "ProblemSeverity": "ERROR",
  "ProblemImpact": "INFRASTRUCTURE",
  "ProblemDetailsJSON": {
    "displayName": "Spark critical ERROR error at TaskSetManager:270",
    "problemId": "-9876543210987654321",
    "rankedEvents": [
      {
        "entityId": "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890",
        "entityName": "spark-master-0",
        "eventType": "ERROR_EVENT",
        "severityLevel": "ERROR",
        "title": "Spark critical ERROR error at TaskSetManager:270",
        "eventDescription": "Spark Cluster critical ERROR error at TaskSetManager:270 on spark-master-0
spark.event_kind=CRITICAL_LOG_EVENT spark.pod_identifier=spark-master-0: 2026-07-20 12:40:29 ERROR TaskSetManager:270
- Task 0 in stage 0.0 failed 4 times; aborting job"
      }
    ],
    "startTime": 1719502426000,
    "endTime": -1
  },
  "ProblemDetailsText": "Spark Cluster critical ERROR error at TaskSetManager:270 on spark-master-0
spark.event_kind=CRITICAL_LOG_EVENT spark.pod_identifier=spark-master-0:\n2026-07-20 12:40:29 ERROR
TaskSetManager:270 - Task 0 in stage 0.0 failed 4 times; aborting job\n\n1 impacted entity: spark-master-0",
  "ImpactedEntities": [
    {
      "entityId": "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890",
      "name": "spark-master-0",
      "type": "CLOUD_APPLICATION_INSTANCE"
    }
  ],
  "ImpactedEntity": "spark-master-0",
  "ProblemURL": "https://<tenant>.apps.dynatrace.com/ui/problems/<problem-id>",
  "State": "OPEN",
  "Tags": "Project:spark-observability",
  "PID": "-9876543210987654321"
}
```

ServiceNow `ResolveApplicationService` binds via `spark.pod_identifier` → pod CI by name → `svc_ci_assoc` Application Service membership (ignores HOST when the pod identifier is present).

Chapter 8. Phase 4 — ServiceNow

ServiceNow Event Management is a three stage process. First, event data is received from external sources, like Dynatrace and an event is created. If the message key of the candidate event matches an existing event, the candidate event is considered a duplicate event, and it will get combined with the earlier event. Then, an event is either promoted to an alert or grouped into an existing alert. Alerts then optionally be promoted to an incident or grouped with an existing incident. Once an incident is created a support team is notified.

In a sense, creating the incident is the goal of event management. The incident represents a call to action to address a critical error condition. However, we need to minimize the noise in the event management process to make it more useful and sustainable for the support team.

At each step in the process there is an opportunity to increase the signal to noise ratio. For example, in the first step, we can deduplicate events from the same source and time window. In the second step, we can group related events into a single alert. In the third step, we can group related alerts into a single incident.

8.1. Event Management

Once an event payload is received, the first step is to create an event object in ServiceNow. The table below shows how the webhook fields map to the event object fields.

Table 11. Webhook payload to em_event field mapping

em_event field	Population
source	SGO-Dynatrace (from URL)
message_key	ProblemID
description	ProblemDetailsText — must include full log line from OpenPipeline event.description
message	ProblemTitle
severity	Mapped from ProblemSeverity (ERROR → 2 Major typical; WARN → 3 Minor if applicable)
type	ERROR or derived from ProblemDetailsJSON ranked event type
node	ImpactedEntity display name (pod name when CLOUD_APPLICATION_INSTANCE)
resource	Log file path or workload.pod_name parsed from ProblemDetailsText or additional_info
cmdb_ci	sys_object_source.target_sys_id for entityId in ImpactedEntities
cmdb_ci_type	cmdb_ci_kubernetes_pod when entity is CLOUD_APPLICATION_INSTANCE; cmdb_ci_appl for PROCESS_GROUP; cmdb_ci_linux_server for HOST fallback
additional_info	JSON: ProblemURL, Tags, ImpactedEntities, full ProblemDetailsJSON, PID
time_of_event	Problem start time from ProblemDetailsJSON
correlation_id	PID when mapped

By design, the SGC seeds the CI of the event from Dynatrace's entityId. For our critical log entries Dynatrace set the dt.source_entity to a cloud application instance or the custom device for Spark clients. For example, "CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890". Then a application oriented (e.g. Spark) Business Rule (BR) overrides the to Spark Client AS (client logs) or pod CI (service logs), else leaves the HOST/empty bind.

After that a ==== CI binding procedure

1. Listener receives POST at inbound_event?source=SGO-Dynatrace.
2. Parser extracts entityId from ImpactedEntities[0] or ProblemDetailsJSON.rankedEvents[0].
3. Query sys_object_source: name=SGO-Dynatrace, id=<entityId>.
4. Set em_event.cmdb_ci and em_event.cmdb_ci_type from the matched row.

8.2. Entity type to CMDB class

Table 12. Dynatrace entityId prefix to CMDB class

Dynatrace entityId prefix	CMDB class	When to expect
CLOUD_APPLICATION_INSTANCE	cmdb_ci_kubernetes_pod	OpenPipeline resolved pod entity (preferred for K8s log paths)
PROCESS_GROUP	cmdb_ci_appl	Log tied to JVM process when pod lookup unavailable
HOST	cmdb_ci_linux_server	Last resort — file tailed on node without pod enrichment

Prerequisites for pod binding:

- SGC Kubernetes Pod import includes pods in the application namespace.
- Pod name in Dynatrace matches Kubernetes metadata.name.
- IRE merge aligns KVA-discovered pod CIs with SGC-imported entities when both exist.
- sys_object_source row exists before the webhook arrives (account for import schedule lag).

8.3. Alert Management

see if there is an existing event with the same message key. If there was no message key, ServiceNow generates a message key from the Source, Node, Type, Resource, and Metric Name fields. The message key (Problem ID) should always be populated by Dynatrace. If the event is a duplicate, it will get grouped into the alert of the existing event. If the duplicate

8.4. Description and alert text

Table 13. Required description and alert text fields

Field	Required content
em_event.description	Line 1: log level and message body from content; Line 2: log.source path; optional ProblemURL appended
em_alert.description	Same as em_event.description (copied by alert rule or inherited on alert create)
em_alert.resource	Pod name or full log file path
em_alert.cmdb_ci	Same infrastructure CI as em_event.cmdb_ci

8.5. SGC and Event Management configuration

Table 14. SGC and Event Management configuration components

Component	Specification
sa_event_field_mapping	ProblemDetailsText → description; ProblemTitle → message; ProblemID → message_key; severity mapping for ERROR and WARN
EM event rule	Mark events Ready when severity and cmdb_ci are populated; optional script normalizes description from additional_info if connector mapping is insufficient
EM alert rule	Create alert from Ready events; copy cmdb_ci and description from event; require cmdb_ci for incident automation
ACL on cmdb_key_value	Roles that run incident scripts must read tags on pod CIs (see install documentation for tag ACL requirements)

8.6. Application and Dynatrace (???? 5)

At Phase 5 the **Observability engineer** typically performs **verification**, not new Dynatrace or application configuration:

1. After a test problem fires, confirm `em_event.source=SGO-Dynatrace` and `em_event.cmdb_ci` is populated (Application Service via `spark.pod_identifier` for Cluster; Spark Client AS via `spark.as_identifier` for Client).
2. If Cluster `cmdb_ci` remains empty or a Linux server, return to Chapter 6 — OneAgent / Cluster OpenPipeline processors and verify `spark.pod_identifier` from `k8s.pod.name` on Container Output, then the entity bind rules.
3. Confirm `sys_object_source` contains a row with `name=SGO-Dynatrace` and `id=<entityId>` from the webhook **before** the event arrives (SGC scheduled import lag is a common root cause of empty CI).
4. Deploy or verify the generic entity bind business rules (entity bind business rules) so `ResolveApplicationService` can bind from `spark.as_identifier` / `spark.pod_identifier`.

No ServiceNow-side changes are required on the monitored **application** (the Java/Spark/Python workload). **No additional Dynatrace** configuration is required at Phase 5 if Chapter 5 and Chapter 6 Dynatrace phases are correct.

Cluster Mode prefers OneAgent Log-module entity context (CAI/PGI) on Container Output;

ServiceNow still binds Application Service via `spark.pod_identifier` (pod CI → `svc_ci_assoc`), not HOST file-tail.

8.7. Create incident bound to Application Service

8.8. Severity policy

Table 15. Automatic incident severity policy by environment

Environment	Automatic incident when
Non-production / test	ERROR and WARN (simplifies validation when ERROR signals are hard to trigger)
Production	ERROR only; WARN remains at alert level or creates a lower-priority task per ITSM policy

Incidents must set **incident.cmdb_ci** to **cmdb_ci_service_discovered** (Application Service), not the infrastructure CI on the alert.

8.9. Layer model

Table 16. Event Management layer model and cmdb_ci classes

Layer	Table	cmdb_ci class
Event	em_event	Infrastructure: cmdb_ci_kubernetes_pod, cmdb_ci_appl, or cmdb_ci_linux_server
Alert	em_alert	Same infrastructure CI as the source event
Incident	incident	cmdb_ci_service_discovered (Application Service)

8.10. Incident correlation

Incident correlation defines how enriched alerts map to `incident.cmdb_ci`. The business rule `em-alert-create-k8s-log-incident` tries the **Client-side** log path first, then the **Service-side** `svc_ci_assoc` membership lookup.

The comparison table summarizes alert CI versus incident CI for each pattern. Full algorithms follow in the adjacent subsections.

Table 17. Incident correlation patterns — alert CI vs Application Service

Pattern	Alert CI	incident.cmdb_ci resolution
Service-side	`cmdb_ci_kubernetes_pod` (preferred) or HOST fallback	Spark Service-Side Log Pattern — `svc_ci_assoc` membership lookup from pod CI
Client-side	HOST or empty (acceptable)	Spark Client-Side Log Pattern — `/client-logs/` → Spark Client AS

8.10.1. Service-side

The **Spark Service-Side Log Pattern** resolves Application Services by looking up the alert's pod CI in the `svc_ci_assoc` membership table maintained by tag-based Service Mapping (see Tag-Based Service Mapping and svc_ci_assoc). A membership row associating the pod CI with a `cmdb_ci_service_discovered` service instance **must** exist before the problem fires.

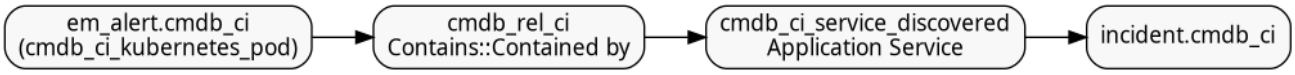

Figure 8. Incident correlation — Spark Service-Side Log Pattern

1. `K8sLogPodCiBind` sets `em_event.cmdb_ci` and `em_alert.cmdb_ci` to the **pod CI** whose `name` matches the log path segment (Phase 5).
2. Query `svc_ci_assoc` where `ci_id` = pod CI sys_id and `service_id` is a `cmdb_ci_service_discovered`.
3. Set `incident.cmdb_ci` = the associated service instance sys_id.

Out of scope: host CI binding without pod rebind, process group CI, `cmdb_rel_ci` ownership edges (none are created in this model).

8.10.2. Client-side

The **Spark Client-Side Log Pattern** maps log paths to a single logical Application Service. Client-mode driver logs **do not** correspond to a `cmdb_ci_kubernetes_pod`; incident automation **must not** depend on alert CI or `svc_ci_assoc` membership.

Figure 9. Incident correlation — Spark Client-Side Log Pattern

1. Scan `em_alert.description`, `resource`, and `node` for the substring `/client-logs/`.
2. Look up Application Service `Spark Client` by `name` (see CSDM identifier lookup).
3. Set `incident.cmdb_ci` to that Application Service sys_id.
4. `K8sLogPodCiBind` skips path segment `spark-client` when parsing pod names.

8.11. CSDM identifier lookup

CSDM `identifier` in `*.csdm.yaml` is the logical machine key embedded in log paths and optional Dynatrace tags. The table below shows which CMDB fields are reliable for incident lookup on optimizincdemo1.

Table 18. CSDM identifier lookup fields on Application Service

Purpose	Mechanism	Queryable field on `cmdb_ci_service_discovered`
Manual / logical AS documentation	CSDM `identifier: spark-client` in log path contract	`identifier` column — often **not populated** on optimizincdemo1

Purpose	Mechanism	Queryable field on `cmdb_ci_service_discovered`
Display / incident lookup	CSDM `name` (e.g. `Spark Client`)	`name` — reliable for `resolveFromSparkClientLogPath()`
Tag-based membership	`servicenow.com/service-instance` = display name (K8s/Docker workloads only)	Not applicable to Spark Client — the client is not a managed CI; binding uses the log-path contract

Spark Client AS **must** have `service_status=operational` (promoted by `csdm/deploy.yml` → `ensure_host_as_operational.yml`).

Listing 25. Verify Spark Client Application Service operational status

```
cd ansible
ansible-playbook -i inventory.yml playbooks/servicenow/incident/verify_spark_client_as.yml \
  -e @../vars/secrets.yaml
```

8.12. Davis event identity vs problem merge

Dynatrace can appear to "bundle" unrelated log lines when `event.name` does not include the intended split key. Reports with the same `event.name` + `event.type` + `dt.source_entity` refresh one Davis event; `event.unique_identifier` **alone does not split that identity**.

Lab approach:

- Custom log source sets `spark.mode`, `spark.driver.instance` or `spark.pod_identifier`, `spark.as_identifier` (Client), and `spark.log.path`. OpenPipeline parses `spark.log.class` / `spark.log.line`.
- Davis `event.name` / `event.unique_identifier` use `{spark.mode}` + **class:line** so each call site is a distinct event on the OneAgent **HOST** `dt.source_entity`.
- `dt.davis.is_merging_allowed=false` so different class:line events are not HOST-bundled; same class:line across drivers already shares one event via `event.name`.
- ServiceNow binds Client alerts via `/client-logs/` or Davis `Client error at …` text to Application Service **Spark Client**; Cluster alerts use the pod's `svc_ci_assoc` membership.

Worker→multi-client cascading failures remain a topology case; this pipeline does not force-correlate unrelated History Server vs Driver call sites beyond what Davis merge decides.

8.13. ServiceNow incident automation specification

Table 19. ServiceNow incident automation components

Component	Specification
Script Include: `ResolveApplicationService`	Service-Side: `svc_ci_assoc` membership from pod CI to Application Service (tag-based Service Mapping). Spark client: `resolveFromSparkClientLogPath()` when alert text contains `/client-logs/` → Application Service **Spark Client**

Component	Specification
Script Include: `K8sLogPodCiBind`	Rebind `em_event` / `em_alert` from initial SGO HOST binding to `cmdb_ci_kubernetes_pod` when log path segment matches a discovered pod `name`; **skip** segment `spark-client`; propagate binding to alerts sharing `message_key`
EM event rule	Mark Spark log events **Ready** when `source=SGO-Dynatrace`, severity matches policy, and `cmdb_ci` is set
EM alert rule	Create alert from Ready events; copy `cmdb_ci`, `description`, `resource`, `node` from event
Business rule: `em-event-bind-k8s-log-pod-ci`	**Before** insert/update on `em_event`; order **5000**; enriches `cmdb_ci`, `resource`, `message_key` via `K8sLogPodCiBind` (must run before EM Ready/alert rules persist the row)
Business rule: `em-event-propagate-k8s-log-pod-ci`	**After** insert/update on `em_event`; order **5005**; copies pod binding from event to related `em_alert` rows sharing `message_key` (runs after the event row is saved)
Business rule: `em-alert-bind-k8s-log-pod-ci`	**Before** insert/update on `em_alert`; order **5010**; same pod/path enrichment when alert is created or updated directly (covers SGC webhook path that skips a separate bind on the event)
Business rule: `em-alert-create-k8s-log-incident`	**After** insert/update on `em_alert`; order **5020**; tries spark-client path first, else Service-Side `svc_ci_assoc` lookup; creates or correlates incident, sets **`em_alert.incident`**
Severity filter	Non-prod: `em_alert.severity ⇐ 3` (ERROR and WARN); Production: `em_alert.severity = 2` (ERROR only)
Duplicate suppression	Correlate open incidents by Application Service + short description prefix `Event Log`; use `ProblemID` / `message_key` for event lifecycle

8.14. Business rule order and rationale

Business rules on `em_event` and `em_alert` run in **order** within the same **when** phase (`before` or `after`). Lower order runs first.

Table 20. Business rule order on em_event and em_alert

Order	Rule	Why this position
5000	`em-event-bind-k8s-log-pod-ci` (before)	Parse log path and set `cmdb_ci` / `resource` / `message_key` **before** the row is written. EM event rules that mark the event Ready and copy fields to alerts depend on these values being present on insert.
5005	`em-event-propagate-k8s-log-pod-ci` (after)	After the event is saved, push binding to sibling `em_alert` rows. Must be **after** (not before) because it updates **other** records.
5010	`em-alert-bind-k8s-log-pod-ci` (before)	Same enrichment for alerts created or updated by the SGC webhook / EM alert rule without going through a separate event bind cycle.
5020	`em-alert-create-k8s-log-incident` (before)	Needs enriched `resource` and log-path context from 5000/5010 in the **same** transaction. Creates the incident and sets `em_alert.incident` before the alert row is committed. Using **after** + `current.update()` was unreliable for REST-driven updates.

8.15. Alert to incident linkage (`em_alert.incident`)

On **optimizincdemo1**, Event Management links an alert to its incident through `em_alert.incident` (reference to the incident record). That field populates the incident **Alerts** tab. Some instances expose the same reference as `em_alert.task`; this build uses `incident` **only** — do not reference `task`.

Creating an incident with `GlideRecord('incident').insert()` alone does **not** set this link — automation must assign `current.incident = <incident_sys_id>` on the alert in the **before** business rule.

Incident short description format: `Event Log WARN` or `Event Log ERROR` (parsed from alert description).

8.16. Problem and alert lifecycle (OPEN / RESOLVED)

Spark/Log4j does **not** emit explicit "clear" or "recovery" log lines for this use case. Closure is driven by **Dynatrace problem lifecycle**, not application log content:

1. OneAgent tails application log files on the **host node** (see Log emission).
2. OpenPipeline Davis turns qualifying **ERROR** / **WARN** lines into Davis events; repeated lines merge into one **problem** (`dt.davis.event_timeout: 15m`).
3. When matching log activity stops, Dynatrace **auto-closes** the problem after the timeout.
4. The brooks-lab problem notification has `notifyClosedProblems: true`, so Dynatrace POSTs a **RESOLVED** webhook with the same `ProblemID` as the OPEN post.
5. SGO maps `message_key` = `ProblemID`, so ServiceNow **updates the same** `em_event` (does not create a duplicate).
6. Event Management then transitions the linked **alert** (and optionally the **incident**) according to configured EM rules when the event state/resolution changes.

Verify RESOLVED handling: `em_event` where `source=SGO-Dynatrace` and `resolution_code` is set after problems clear; linked `em_alert.state` should move to **Closed** when EM auto-close rules are configured.

8.17. OpenPipeline and custom-source attributes (generic Log4j pattern)

Attributes added by automation (not Spark-specific semantics):

Table 21. OpenPipeline and custom log source attributes

Attribute	Source	Purpose
`spark.log.path`	Custom log source enrichment (`${0}`)	Literal file path of the log line
`spark.mode`	Custom log source	`Client` or `Cluster`
`spark.pod_identifier`	Cluster path segment	Must equal `cmdb_ci_kubernetes_pod.name`

Attribute	Source	Purpose
`spark.instance`	Client OpenPipeline	`<host.name>:<driver-instance>` under `/mnt/spark/client-logs/`
`spark.as_identifier`	Client custom source / OpenPipeline	Application Service `identifier` (`spark-client`) for SN bind
`event.type`	OpenPipeline Davis / metric event template	`ERROR_EVENT` (Spark log); `CUSTOM_ALERT` (host CPU metric) — Dynatrace enum
`spark.event_kind`	OpenPipeline Davis / metric event template	`CRITICAL_LOG_EVENT` (Spark log); `CPU_EVENT` (host CPU metric)
`em_event.type` / `em_alert.type`	ServiceNow `ResolveApplicationService.applySparkEventTypeRename`	Remapped to `CRITICAL_LOG_EVENT` / `CPU_EVENT` from `spark.event_kind` (built-in `CPU_SATURATED` unchanged)
`event.name`	OpenPipeline Davis	`Spark critical {loglevel} error at {spark.log.class}:{spark.log.line}`
`event.unique_identifier`	OpenPipeline Davis	`Spark Client
Cluster Critical {loglevel} error {spark.log.class}:{spark.log.line}`	`event.description`	OpenPipeline Davis

The Pod **Kubernetes label** (`servicenow.com/service-instance`) is the CMDB/service-mapping contract — not a Dynatrace tag. See pod labels.

Cluster/Client Davis problems keep OneAgent **HOST** as `dt.source_entity` inside the Spark Observability management zone. ServiceNow `ResolveApplicationService` binds Cluster alerts via `spark.pod_identifier` (pod CI → `svc_ci_assoc` AS) and Client alerts via `spark.as_identifier` to Application Service **Spark Client**, ignoring HOST for those paths.

8.18. Event Management rules (manual configuration)

Configure under **Event Management → Rules**:

Table 22. Event Management rules for SGO-Dynatrace Spark logs

Rule	Condition and action
Event rule (K8s application logs)	**When:** `em_event` insert/update; **Filter:** `source=SGO-Dynatrace` `severity<=3`^[3]`cmdb_ciISNOTEMPTY`; **Action:** set state to **Ready** (or use existing Ready rule scoped to SGO-Dynatrace)
Alert rule (K8s application logs)	**When:** Ready event; **Filter:** `source=SGO-Dynatrace`; **Action:** create/update alert; copy `cmdb_ci`, `description`, `resource`, `node`, `message_key` from event

Scope Spark-specific business rules with `source=SGO-Dynatrace` so legacy `source=Dynatrace` events are unaffected during connector migration (see Legacy and SGO-Dynatrace coexistence).

8.19. Script Includes

Deploy from `servicenow/integrations/incident/`. Global scope, **Active** = true. Ansible: `ansible/playbooks/servicenow/incident/deploy.yml`.

Each Script Include below summarizes **purpose**, **inputs**, **outputs / side effects**, and how it fits the business rules. Source of truth is the `.si.js` file in the repository.

8.19.1. ResolveApplicationService

Purpose: Resolve `cmdb_ci_service_discovered` (Application Service) sys_id for incident binding.

Methods:

- `resolveFromInfrastructureCi(ciSysId)` — **Spark Service-Side Log Pattern.** Queries `svc_ci_assoc` where `ci_id` = pod CI and `service_id` is a `cmdb_ci_service_discovered` (membership maintained by tag-based Service Mapping). Returns the service instance sys_id or `null`.
- `resolveFromSparkClientLogPath(text)` — **Spark Client-Side Log Pattern.** When `text` contains `/client-logs/`, looks up Application Service **Spark Client** by `identifier=spark-client`, then fallback `name=Spark Client`.

Inputs: Workload CI sys_id (Service-side) or alert description/resource text (Client-side).

Outputs: Application Service sys_id for `incident.cmdb_ci`; no CMDB writes.

Side effects: None (read-only GlideRecord queries).

Source: `servicenow/integrations/incident/ResolveApplicationService.si.js`

8.19.2. K8sLogPodCiBind

Purpose: Rebind `em_event` and `em_alert` from HOST or empty CI to `cmdb_ci_kubernetes_pod` when the log path directory segment matches a discovered pod name (OpenPipeline often leaves problems on HOST).

Methods:

- `resolvePodName(description, resource, node)` — Scans concatenated fields for `/⋯/logs/<segment>/`; skips `spark-client` and wildcards; verifies pod exists in CMDB.
- `applyPodBinding(gr)` — Sets `cmdb_ci`, `node`, `resource`, `message_key` prefix `K8sLog-{pod}-`, and `message` to `Event Log WARN/ERROR` when generic. Returns `true` on success. Caller must `update()`.
- `propagateEventToAlerts(eventGr)` — Copies pod binding from event to related alerts sharing `message_key`.

Inputs: `em_event` or `em_alert` GlideRecord.

Outputs: Mutated fields on current row; optional `em_alert.update()` on related rows.

Source: servicenow/integrations/incident/K8sLogPodCiBind.si.js

8.20. Business rules (prescriptive configuration)

Create under **System Definition → Business Rules**. All three run **After** insert and update. Filter conditions must include `source=SGO-Dynatrace` so legacy connectors are not modified.

Table 23. Prescriptive K8s log business rules

Name	Table	Order	Filter condition
em-event-bind-k8s-log-pod-ci	em_event	5000	source=SGO-Dynatrace
em-alert-bind-k8s-log-pod-ci	em_alert	5010	source=SGO-Dynatrace
em-alert-create-k8s-log-incident	em_alert	5100	source=SGO-Dynatrace^severity⇐3

8.20.1. em-event-bind-k8s-log-pod-ci

Listing 26. Business rule script — em-event-bind-k8s-log-pod-ci

```
(function executeRule(current, previous /*null on insert*/) {
  var binder = new K8sLogPodCiBind();
  if (!binder.applyPodBinding(current)) {
    return;
  }

  current.update();
  binder.propagateEventToAlerts(current);
})(current, previous);
```

8.20.2. em-alert-bind-k8s-log-pod-ci

Listing 27. Business rule script — em-alert-bind-k8s-log-pod-ci

```
(function executeRule(current, previous /*null on insert*/) {
  var binder = new K8sLogPodCiBind();
  if (!binder.applyPodBinding(current)) {
    return;
  }

  current.update();
})(current, previous);
```

8.20.3. em-alert-create-k8s-log-incident

Creates or correlates an incident on the Application Service, then sets `current.task` so the alert appears on the incident **Alerts** tab.

Listing 28. Business rule script — em-alert-create-k8s-log-incident

```
(function executeRule(current, previous /*null on insert*/) {
  if (current.source.toString() !== 'SGO-Dynatrace') {
    return;
  }
```

```
if (!current.task.nil()) {
  return;
}

var severity = parseInt(current.severity, 10);
if (isNaN(severity) || severity > 3) {
  return;
}

var desc = current.description.toString();
if (
  desc.indexOf('Spark ') === -1 &&
  desc.indexOf('spark-app.log') === -1 &&
  desc.indexOf('/mnt/spark/logs/') === -1
) {
  return;
}

if (current.cmdb_ci.nil()) {
  return;
}

var resolver = new ResolveApplicationService();
var asSysId = resolver.resolveFromInfrastructureCi(current.cmdb_ci.toString());
if (!asSysId) {
  return;
}

var levelMatch = desc.match(/\b(ERROR|WARN)\b/);
var logLevel = levelMatch
  ? levelMatch[1]
  : severity <= 2
    ? 'ERROR'
    : 'WARN';
var shortDesc = 'Event Log ' + logLevel;
var shortDescPrefix = 'Event Log';

function linkAlertToIncident(incSysId) {
  var alertGr = new GlideRecord('em_alert');
  if (!alertGr.get(current.sys_id)) {
    return;
  }
  alertGr.task = incSysId;
  alertGr.setWorkflow(false);
  alertGr.update();
}

var backfillInc = new GlideRecord('incident');
backfillInc.addQuery('active', true);
backfillInc.addQuery('short_description', 'STARTSWITH', shortDescPrefix);
backfillInc.addEncodedQuery('cmdb_ciISEMPTY');
backfillInc.orderBy('sys_created_on');
backfillInc.setLimit(1);
backfillInc.query();
if (backfillInc.next()) {
  backfillInc.cmdb_ci = asSysId;
  backfillInc.short_description = shortDesc;
  backfillInc.work_notes =
    'Set Configuration item from alert ' +
    current.number +
    ' (Application Service resolved from pod CI).';
  backfillInc.update();
  linkAlertToIncident(backfillInc.sys_id);
  return;
}

var openInc = new GlideRecord('incident');
openInc.addQuery('cmdb_ci', asSysId);
```

```
  openInc.addQuery('active', true);
  openInc.addQuery('short_description', 'STARTSWITH', shortDescPrefix);
  openInc.setLimit(1);
  openInc.query();
  if (openInc.next()) {
    openInc.work_notes =
      'Correlated alert ' +
      current.number +
      ': ' +
      current.description.toString().substring(0, 500);
    openInc.update();
    linkAlertToIncident(openInc.sys_id);
    return;
  }

  var inc = new GlideRecord('incident');
  inc.initialize();
  inc.cmdb_ci = asSysId;
  inc.short_description = shortDesc;
  inc.description = current.description;
  inc.severity = current.severity;
  inc.impact = 2;
  inc.urgency = 2;
  inc.caller_id = gs.getUserID();
  inc.comments = 'Auto-created from Event Management alert ' + current.number;
  inc.work_notes = 'Source alert: ' + current.number;
  var incSysId = inc.insert();
  linkAlertToIncident(incSysId);
})(current, previous);
```

8.21. Ansible deployment

Listing 29. Deploy Log-to-Incident ServiceNow automation

```
cd ansible
ansible-playbook -i inventory.yml playbooks/servicenow/incident/deploy.yml \
  -e @../vars/secrets.yaml
```

Playbook path: `ansible/playbooks/servicenow/incident/deploy.yml`. Source files: `servicenow/integrations/incident/*.js`.

8.22. Example resolution (Kubernetes worker pod)

Table 24. Example resolution path for a Kubernetes worker pod log

Phase	Record / relationship
Problem	Davis problem; impacted entity CLOUD_APPLICATION_INSTANCE-A1B2C3D4E5F67890
Event	em_event; cmdb_ci → cmdb_ci_kubernetes_pod myapp-worker-7d4f9b-xk2lm
Alert	em_alert; same pod CI; description contains full ERROR log line
Tag on pod CI	cmdb_key_value: servicenow.com/service-instance=Myapp-Worker-Zone-A
SM membership	svc_ci_assoc: pod CI is a member of Application Service Myapp Worker Zone A (tag-based Service Mapping)
Incident	incident.cmdb_ci → Application Service Myapp Worker Zone A (via Service-Side svc_ci_assoc lookup)

8.23. Application and Dynatrace (Phase 5)

No configuration on the application or Dynatrace side. Dynatrace problem tags (Project, Environment) do not replace CSDM tags on pod CIs for incident binding.